COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010CVPE	**SECTION:**	CPE21S4
DATE PERFORMED:	08/07/2026	**DATE SUBMITTED:**	08/07/2026
NAME:	KEITH NICOLE SILVOZA	**INSTRUCTOR:**	JIMLORD QUEJADO

ACTIVITY NO. 5.1 QUEUES

1. PROCEDURE OUTPUT

What procedures did you perform?

Intended Learning Outcomes Addressed

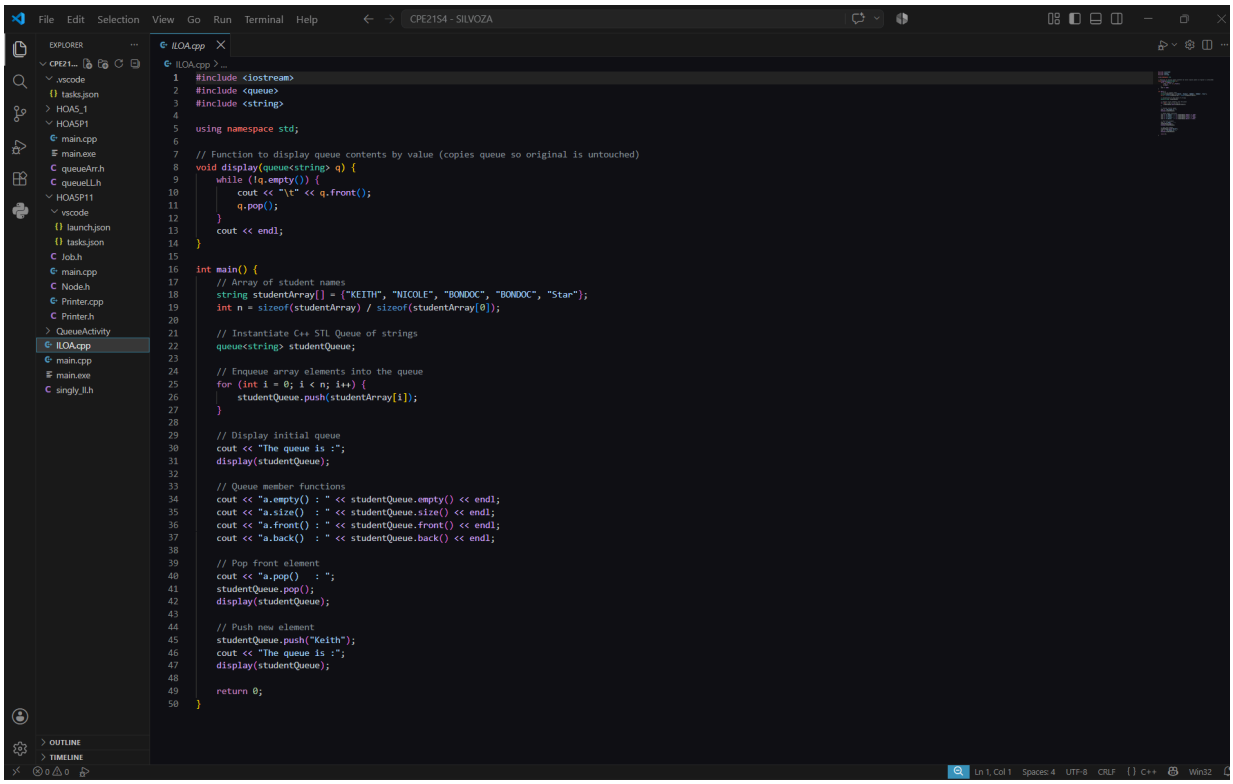
At the end of the activity the students will be able to:

- Create queue using C++ STL
- Develop C++ code that uses both arrays and linked lists to create a stack
- Solve problems using an implementation of stack

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO-A



```cpp
#include <iostream>
#include <queue>
#include <string>

using namespace std;

// Function to display queue contents by value (copies queue so original is untouched)
void display(queue<string> q) {
    while (!q.empty()) {
        cout << "\t" << q.front();
        q.pop();
    }
    cout << endl;
}

int main() {
    // Array of student names
    string studentArray[] = {"KEITH", "NICOLE", "BONDOC", "BONDOC", "Star"};
    int n = sizeof(studentArray) / sizeof(studentArray[0]);

    // Instantiate C++ STL Queue of strings
    queue<string> studentQueue;

    // Enqueue array elements into the queue
    for (int i = 0; i < n; i++) {
        studentQueue.push(studentArray[i]);
    }

    // Display initial queue
    cout << "The queue is :";
    display(studentQueue);

    // Queue member functions
    cout << "a.empty() : " << studentQueue.empty() << endl;
    cout << "a.size()  : " << studentQueue.size() << endl;
    cout << "a.front() : " << studentQueue.front() << endl;
    cout << "a.back()  : " << studentQueue.back() << endl;

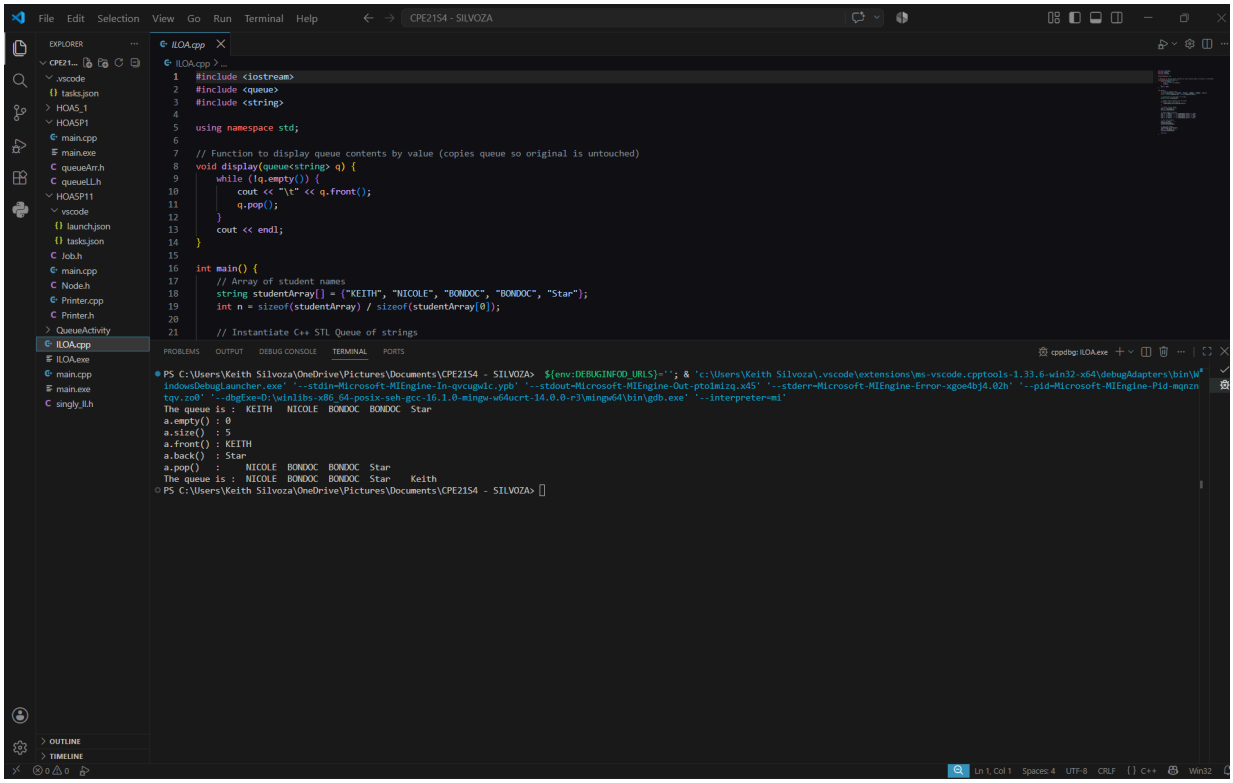
    // Pop front element
    cout << "a.pop()   : ";
    studentQueue.pop();
    display(studentQueue);

    // Push new element
    studentQueue.push("Keith");
    cout << "The queue is :";
    display(studentQueue);

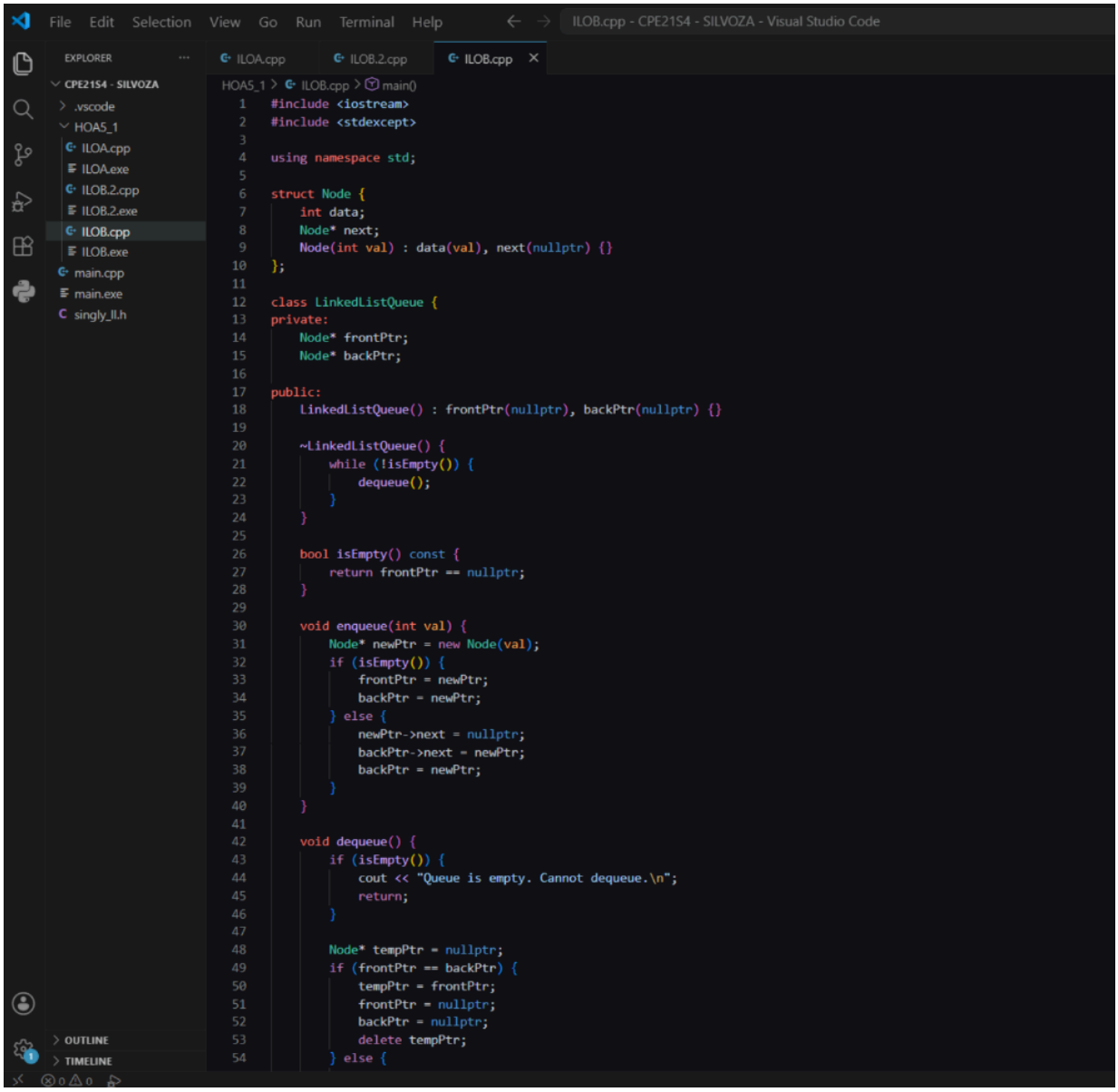
    return 0;
}
```

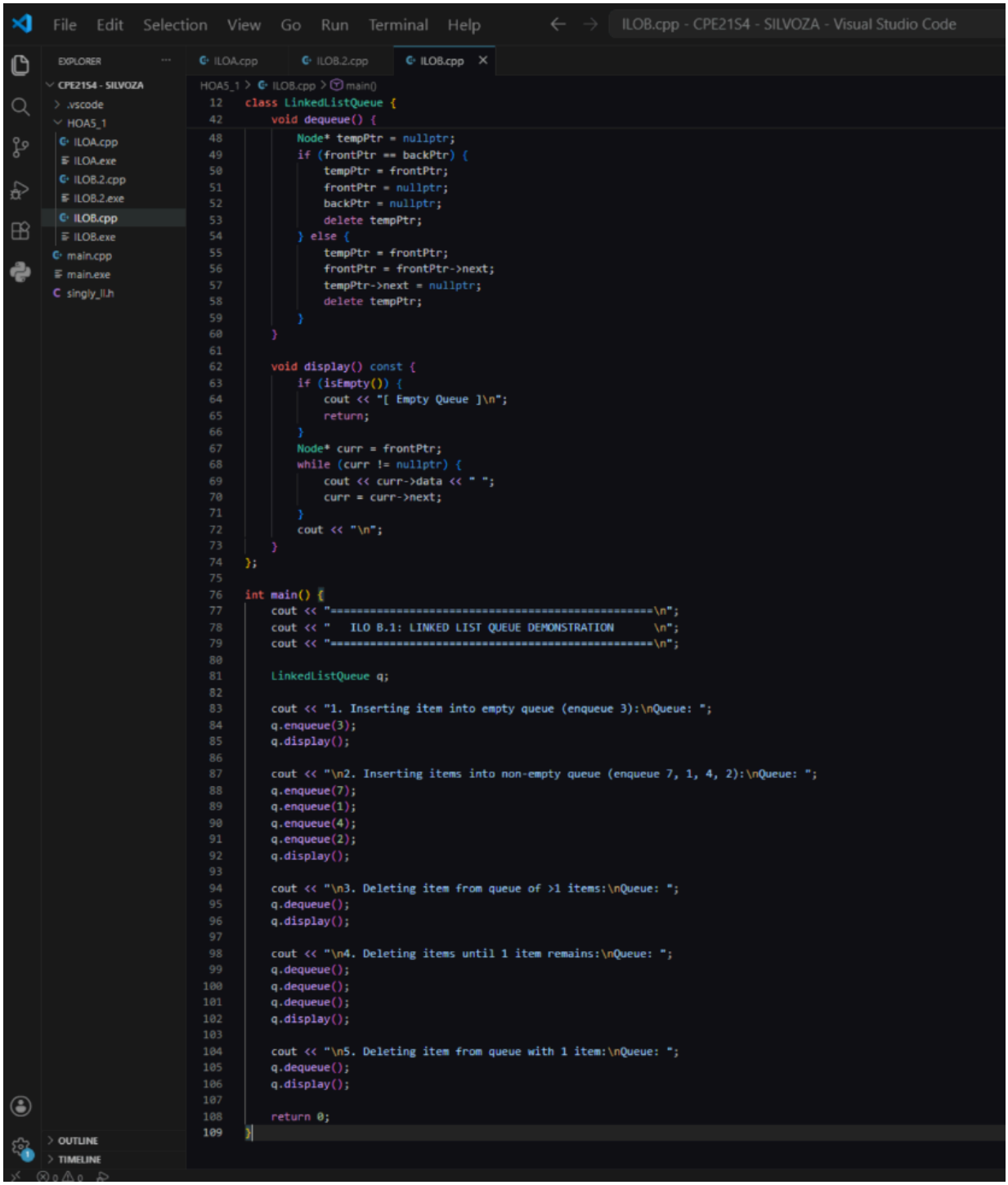
COMPUTER ENGINEERING

QUEZON CITY



ILO-B


File Edit Selection View Go Run Terminal Help
ILOB.cpp - CPE21S4 - SILVOZA - Visual Studio Code
EXPLORER
CPE21S4 - SILVOZA
.vscode
HOA5_1
ILOA.cpp
ILOA.exe
ILOB.2.cpp
ILOB.2.exe
ILOB.cpp
ILOB.exe
main.cpp
main.exe
singly_ll.h
ILOA.cpp
ILOB.2.cpp
ILOB.cpp
HOA5_1 > ILOB.cpp > main()
#include <iostream>
#include <stdexcept>
using namespace std;
struct Node {
int data;
Node* next;
Node(int val) : data(val), next(nullptr) {}
};
class LinkedListQueue {
private:
Node* frontPtr;
Node* backPtr;
public:
LinkedListQueue() : frontPtr(nullptr), backPtr(nullptr) {}
~LinkedListQueue() {
while (!isEmpty()) {
dequeue();
}
}
bool isEmpty() const {
return frontPtr == nullptr;
}
void enqueue(int val) {
Node* newPtr = new Node(val);
if (isEmpty()) {
frontPtr = newPtr;
backPtr = newPtr;
} else {
newPtr->next = nullptr;
backPtr->next = newPtr;
backPtr = newPtr;
}
}
void dequeue() {
if (isEmpty()) {
cout << "Queue is empty. Cannot dequeue.\n";
return;
}
Node* tempPtr = nullptr;
if (frontPtr == backPtr) {
tempPtr = frontPtr;
frontPtr = nullptr;
backPtr = nullptr;
delete tempPtr;
} else {
OUTLINE
TIMELINE




```cpp
class LinkedListQueue {
    void dequeue() {
        Node* tempPtr = nullptr;
        if (frontPtr == backPtr) {
            tempPtr = frontPtr;
            frontPtr = nullptr;
            backPtr = nullptr;
            delete tempPtr;
        } else {
            tempPtr = frontPtr;
            frontPtr = frontPtr->next;
            tempPtr->next = nullptr;
            delete tempPtr;
        }
    }

    void display() const {
        if (isEmpty()) {
            cout << "[ Empty Queue ]\n";
            return;
        }
        Node* curr = frontPtr;
        while (curr != nullptr) {
            cout << curr->data << " ";
            curr = curr->next;
        }
        cout << "\n";
    }
};

int main() {
    cout << "================================================\n";
    cout << "   ILO B.1: LINKED LIST QUEUE DEMONSTRATION      \n";
    cout << "================================================\n";

    LinkedListQueue q;

    cout << "1. Inserting item into empty queue (enqueue 3):\nQueue: ";
    q.enqueue(3);
    q.display();

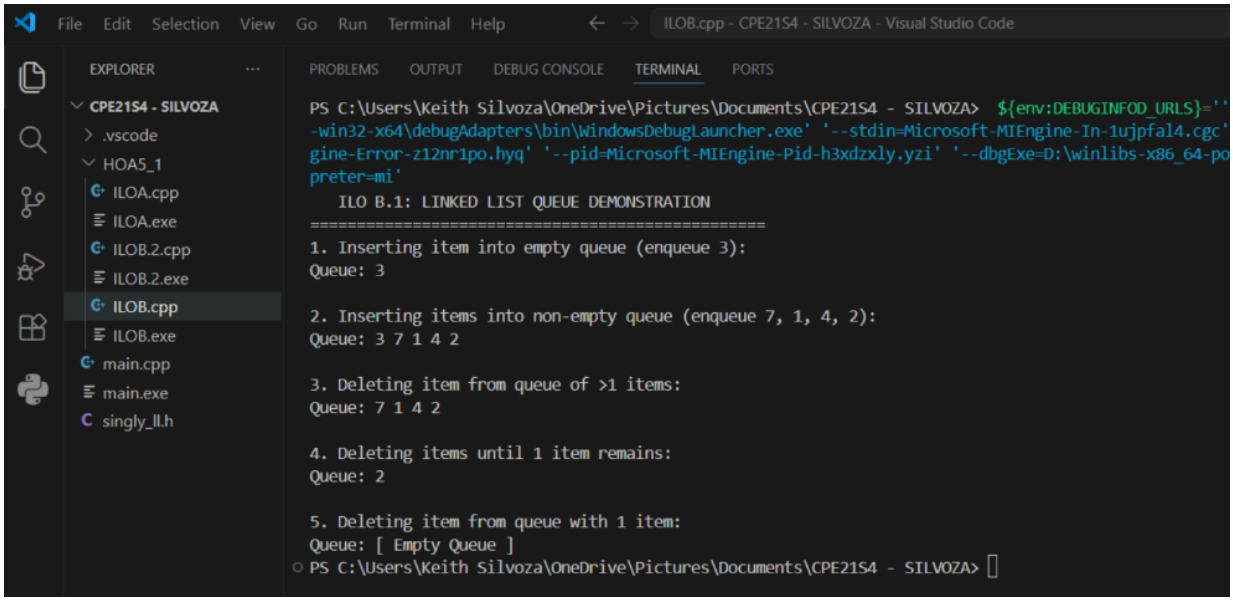
    cout << "\n2. Inserting items into non-empty queue (enqueue 7, 1, 4, 2):\nQueue: ";
    q.enqueue(7);
    q.enqueue(1);
    q.enqueue(4);
    q.enqueue(2);
    q.display();

    cout << "\n3. Deleting item from queue of >1 items:\nQueue: ";
    q.dequeue();
    q.display();

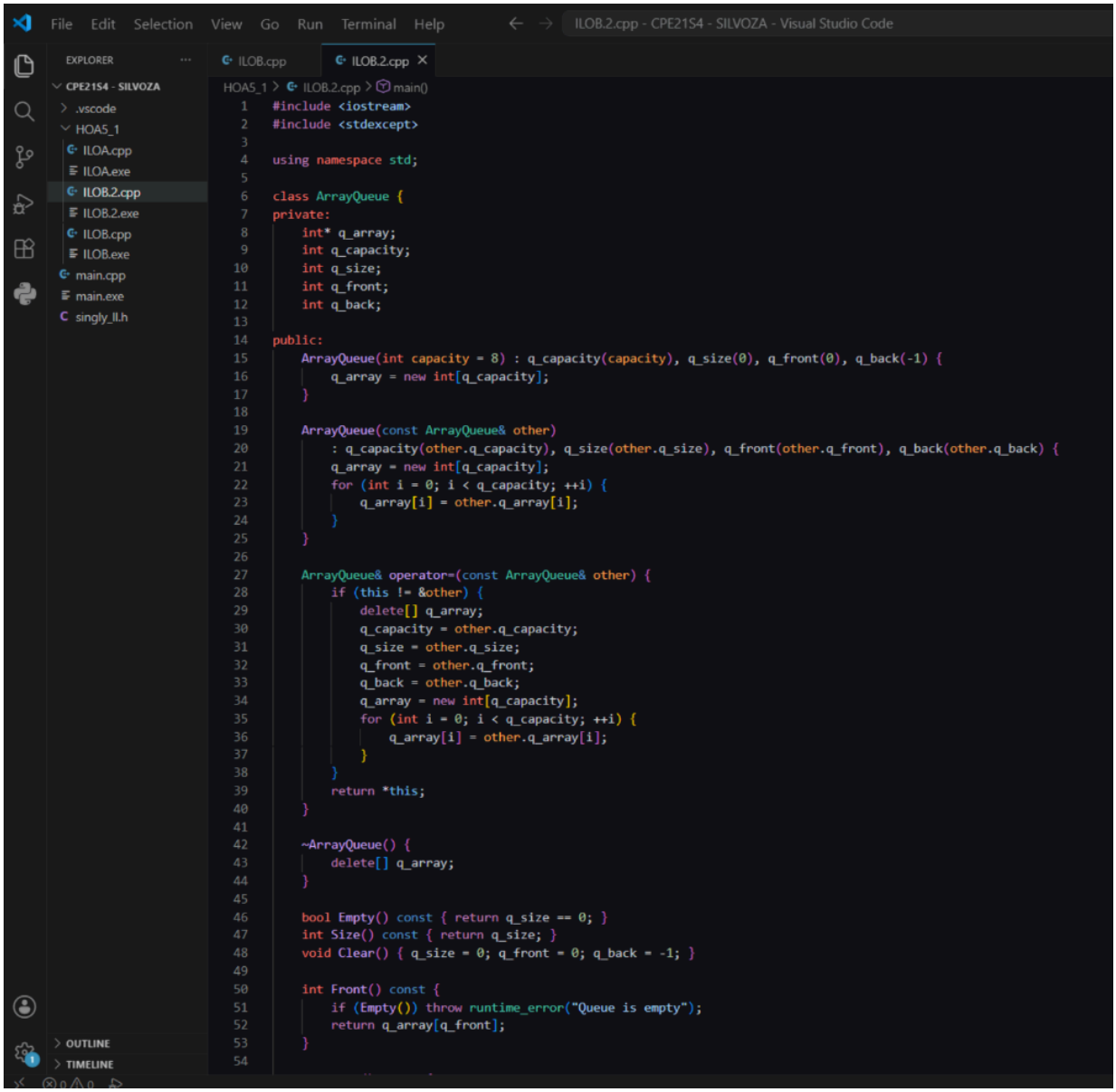
    cout << "\n4. Deleting items until 1 item remains:\nQueue: ";
    q.dequeue();
    q.dequeue();
    q.dequeue();
    q.display();

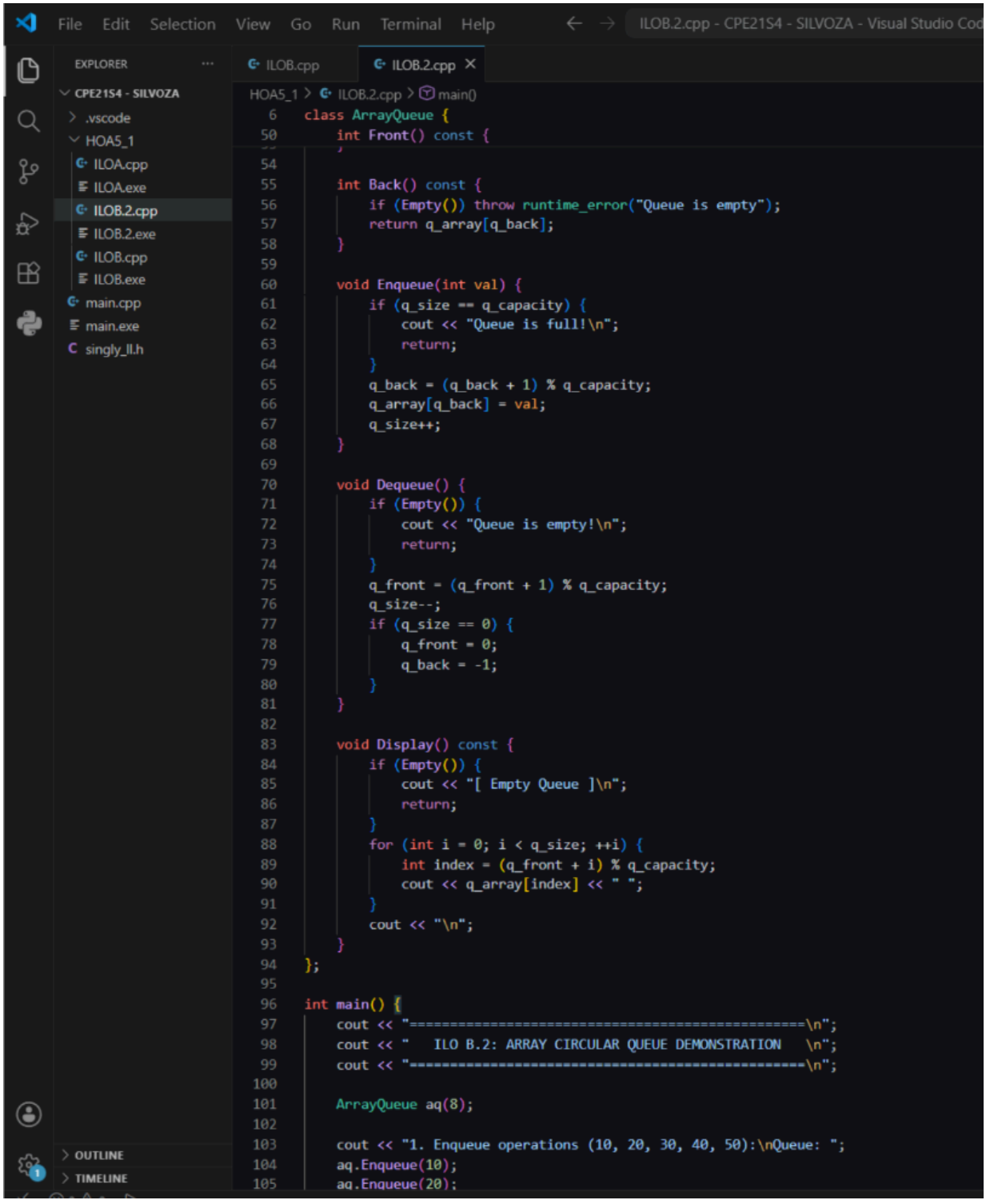
    cout << "\n5. Deleting item from queue with 1 item:\nQueue: ";
    q.dequeue();
    q.display();

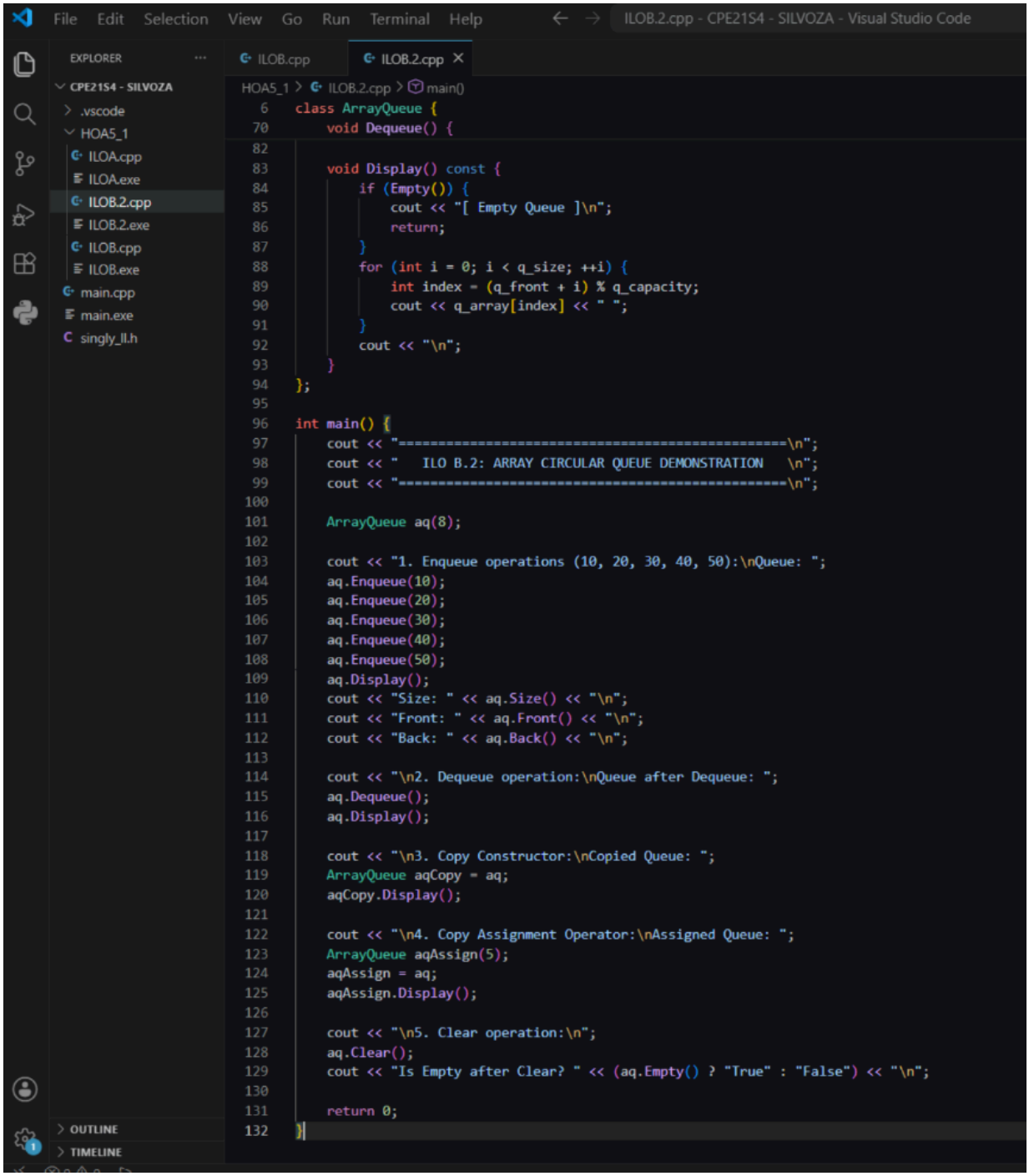
    return 0;
}
```

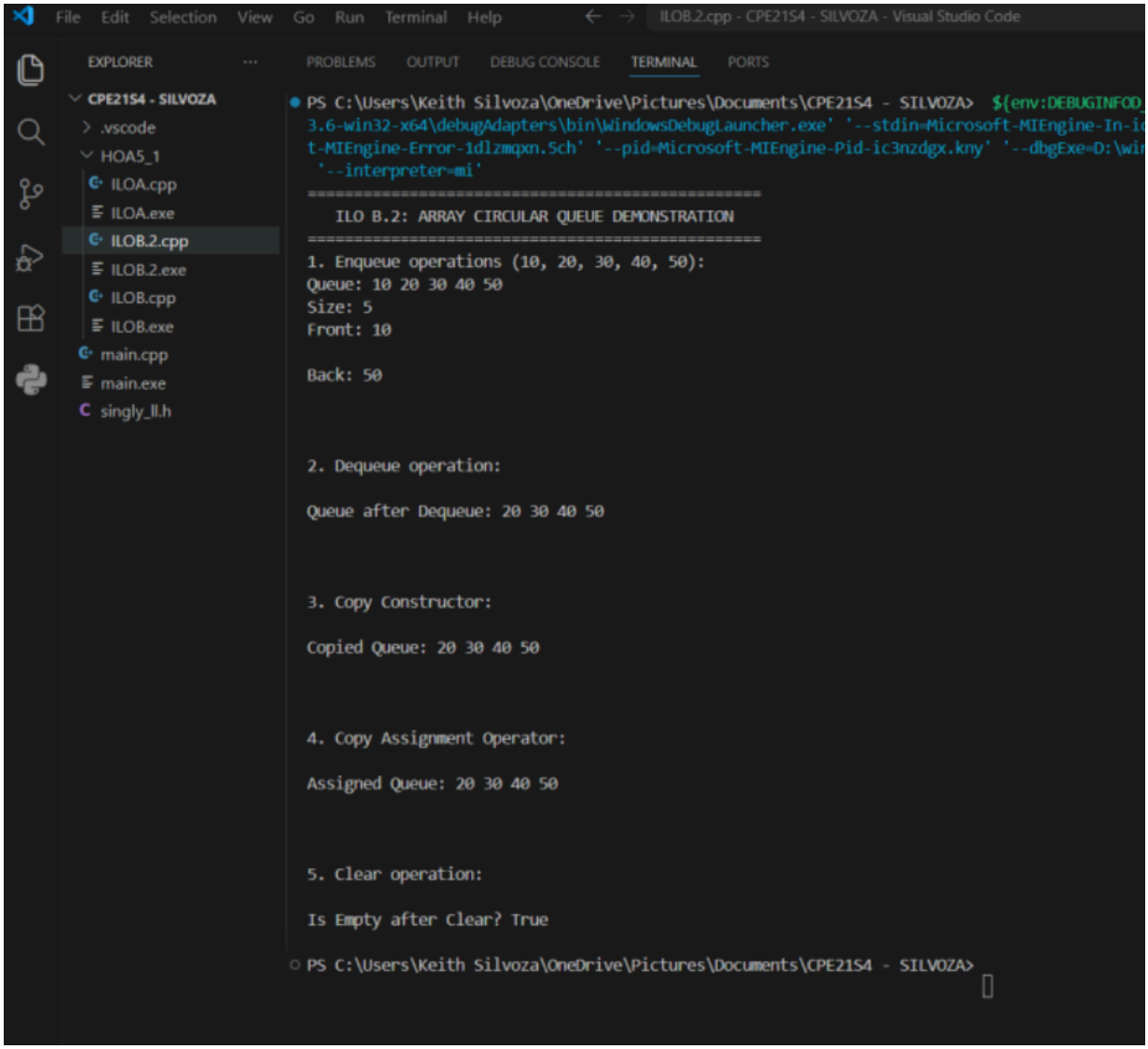

File Edit Selection View Go Run Terminal Help
ILOB.cpp - CPE21S4 - SILVOZA - Visual Studio Code
EXPLORER
CPE21S4 - SILVOZA
.vscode
HOA5_1
ILOA.cpp
ILOA.exe
ILOB.2.cpp
ILOB.2.exe
ILOB.cpp
ILOB.exe
main.cpp
main.exe
singly_ll.h
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Keith Silvoza\OneDrive\Pictures\Documents\CPE21S4 - SILVOZA> ${env:DEBUGINFOD_URLS}=''
-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-1ujpfal4.cgc'
gine-Error-z12nr1po.hyq' '--pid=Microsoft-MIEngine-Pid-h3xdzxly.yzi' '--dbgExe=D:\winlibs-x86_64-po
preter=mi'
ILO B.1: LINKED LIST QUEUE DEMONSTRATION
1. Inserting item into empty queue (enqueue 3):
Queue: 3
2. Inserting items into non-empty queue (enqueue 7, 1, 4, 2):
Queue: 3 7 1 4 2
3. Deleting item from queue of >1 items:
Queue: 7 1 4 2
4. Deleting items until 1 item remains:
Queue: 2
5. Deleting item from queue with 1 item:
Queue: [Empty Queue]
PS C:\Users\Keith Silvoza\OneDrive\Pictures\Documents\CPE21S4 - SILVOZA>

ILO-B.2

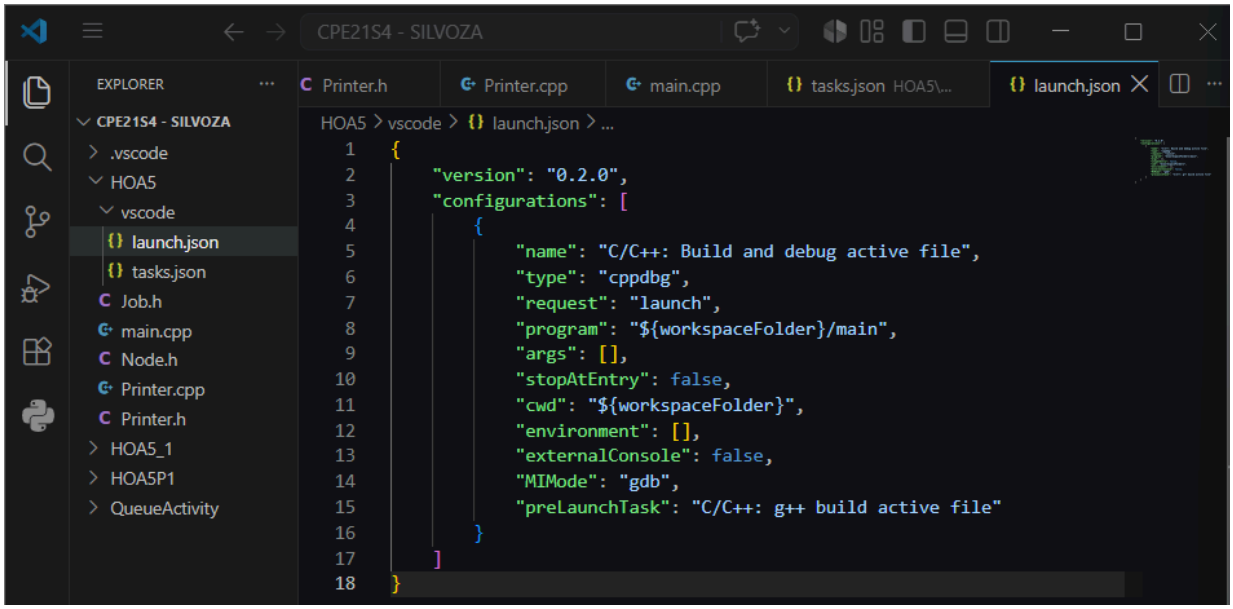

File Edit Selection View Go Run Terminal Help
ILOB.2.cpp - CPE21S4 - SILVOZA - Visual Studio Code
EXPLORER
CPE21S4 - SILVOZA
.vscode
HOA5_1
ILOA.cpp
ILOA.exe
ILOB.2.cpp
ILOB.2.exe
ILOB.cpp
ILOB.exe
main.cpp
main.exe
singly_ll.h
OUTLINE
TIMELINE
ILOB.cpp
ILOB.2.cpp
HOA5_1 > ILOB.2.cpp > main()
#include <iostream>
#include <stdexcept>
using namespace std;
class ArrayQueue {
private:
int* q_array;
int q_capacity;
int q_size;
int q_front;
int q_back;
public:
ArrayQueue(int capacity = 8) : q_capacity(capacity), q_size(0), q_front(0), q_back(-1) {
q_array = new int[q_capacity];
}
ArrayQueue(const ArrayQueue& other)
: q_capacity(other.q_capacity), q_size(other.q_size), q_front(other.q_front), q_back(other.q_back) {
q_array = new int[q_capacity];
for (int i = 0; i < q_capacity; ++i) {
q_array[i] = other.q_array[i];
}
}
ArrayQueue& operator=(const ArrayQueue& other) {
if (this != &other) {
delete[] q_array;
q_capacity = other.q_capacity;
q_size = other.q_size;
q_front = other.q_front;
q_back = other.q_back;
q_array = new int[q_capacity];
for (int i = 0; i < q_capacity; ++i) {
q_array[i] = other.q_array[i];
}
}
return *this;
}
~ArrayQueue() {
delete[] q_array;
}
bool Empty() const { return q_size == 0; }
int Size() const { return q_size; }
void Clear() { q_size = 0; q_front = 0; q_back = -1; }
int Front() const {
if (Empty()) throw runtime_error("Queue is empty");
return q_array[q_front];
}

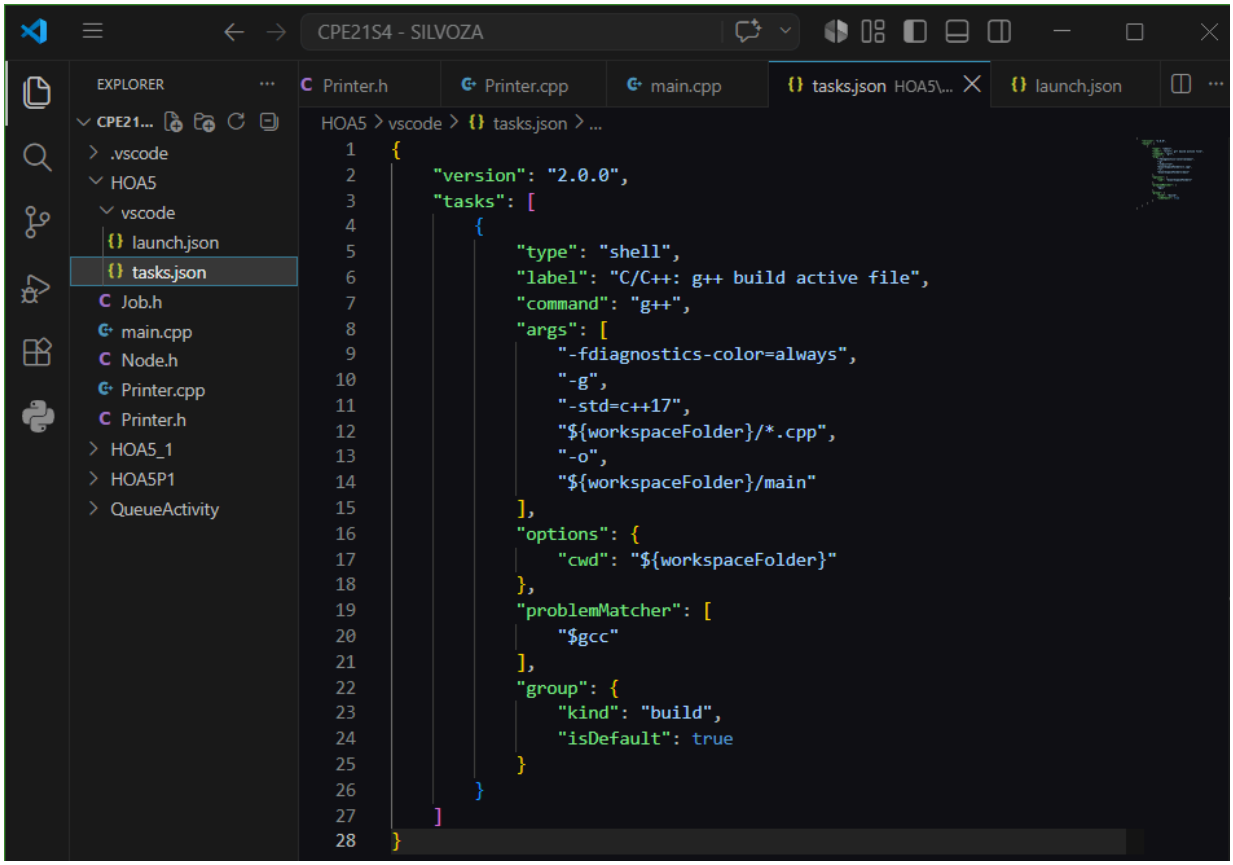

File Edit Selection View Go Run Terminal Help
ILOB.2.cpp - CPE21S4 - SILVOZA - Visual Studio Cod
EXPLORER
CPE21S4 - SILVOZA
.vscode
HOA5_1
ILOA.cpp
ILOA.exe
ILOB.2.cpp
ILOB.2.exe
ILOB.cpp
ILOB.exe
main.cpp
main.exe
singly_ll.h
OUTLINE
TIMELINE
ILOB.cpp
ILOB.2.cpp
HOA5_1 > ILOB.2.cpp > main()
class ArrayQueue {
int Front() const {
int Back() const {
if (Empty()) throw runtime_error("Queue is empty");
return q_array[q_back];
void Enqueue(int val) {
if (q_size == q_capacity) {
cout << "Queue is full!\n";
return;
q_back = (q_back + 1) % q_capacity;
q_array[q_back] = val;
q_size++;
void Dequeue() {
if (Empty()) {
cout << "Queue is empty!\n";
return;
q_front = (q_front + 1) % q_capacity;
q_size--;
if (q_size == 0) {
q_front = 0;
q_back = -1;
void Display() const {
if (Empty()) {
cout << "[Empty Queue]\n";
return;
for (int i = 0; i < q_size; ++i) {
int index = (q_front + i) % q_capacity;
cout << q_array[index] << " ";
cout << "\n";
};
int main() {
cout << "==========\n";
cout << " ILO B.2: ARRAY CIRCULAR QUEUE DEMONSTRATION \n";
cout << "==========\n";
ArrayQueue aq(8);
cout << "1. Enqueue operations (10, 20, 30, 40, 50):\nQueue: ";
aq.Enqueue(10);
aq.Enqueue(20);

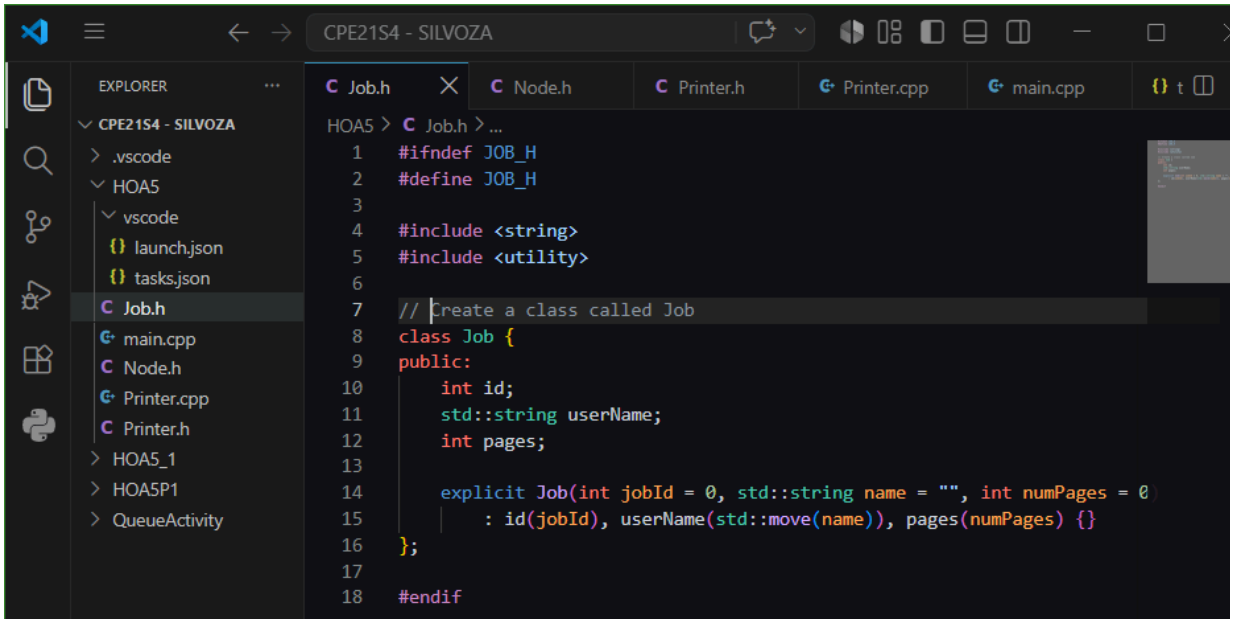

File Edit Selection View Go Run Terminal Help
ILOB.2.cpp - CPE21S4 - SILVOZA - Visual Studio Code
EXPLORER
CPE21S4 - SILVOZA
.vscode
HOA5_1
ILOA.cpp
ILOA.exe
ILOB.2.cpp
ILOB.2.exe
ILOB.cpp
ILOB.exe
main.cpp
main.exe
singly_ll.h
OUTLINE
TIMELINE
ILOB.cpp
ILOB.2.cpp
HOA5_1 > ILOB.2.cpp > main()
class ArrayQueue {
void Dequeue() {
void Display() const {
if (Empty()) {
cout << "[Empty Queue]\n";
return;
}
for (int i = 0; i < q_size; ++i) {
int index = (q_front + i) % q_capacity;
cout << q_array[index] << " ";
}
cout << "\n";
}
};
int main() {
cout << "==========\n";
cout << " ILO B.2: ARRAY CIRCULAR QUEUE DEMONSTRATION \n";
cout << "==========\n";
ArrayQueue aq(8);
cout << "1. Enqueue operations (10, 20, 30, 40, 50):\nQueue: ";
aq.Enqueue(10);
aq.Enqueue(20);
aq.Enqueue(30);
aq.Enqueue(40);
aq.Enqueue(50);
aq.Display();
cout << "Size: " << aq.Size() << "\n";
cout << "Front: " << aq.Front() << "\n";
cout << "Back: " << aq.Back() << "\n";
cout << "\n2. Dequeue operation:\nQueue after Dequeue: ";
aq.Dequeue();
aq.Display();
cout << "\n3. Copy Constructor:\nCopied Queue: ";
ArrayQueue aqCopy = aq;
aqCopy.Display();
cout << "\n4. Copy Assignment Operator:\nAssigned Queue: ";
ArrayQueue aqAssign(5);
aqAssign = aq;
aqAssign.Display();
cout << "\n5. Clear operation:\n";
aq.Clear();
cout << "Is Empty after Clear? " << (aq.Empty() ? "True" : "False") << "\n";
return 0;
}

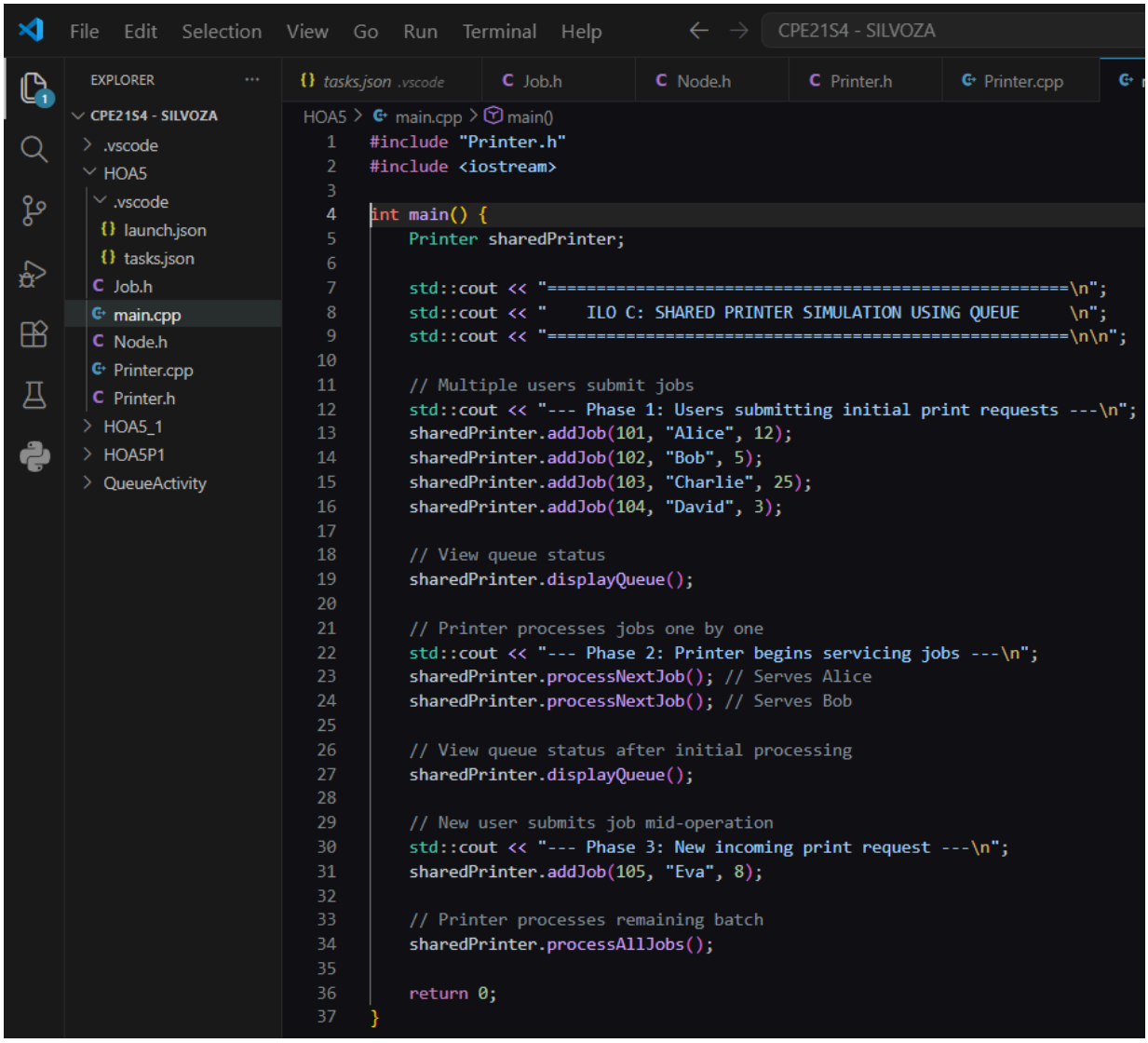


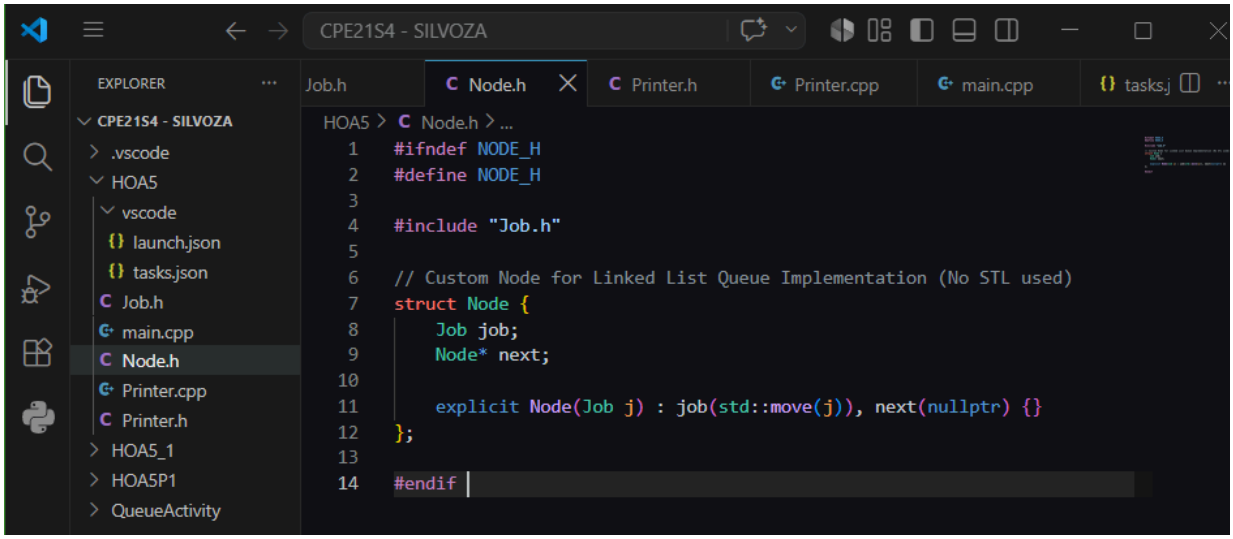
ILO-C / SUPPLEMENTARY


CPE21S4 - SILVOZA
EXPLORER
CPE21S4 - SILVOZA
.vscode
HOA5
vscode
launch.json
tasks.json
Job.h
main.cpp
Node.h
Printer.cpp
Printer.h
HOA5_1
HOA5P1
QueueActivity
Printer.h
Printer.cpp
main.cpp
tasks.json HOA5\...
launch.json
HOA5 > vscode > launch.json > ...
{
"version": "0.2.0",
"configurations": [
{
"name": "C/C++: Build and debug active file",
"type": "cppdbg",
"request": "launch",
"program": "${workspaceFolder}/main",
"args": [],
"stopAtEntry": false,
"cwd": "${workspaceFolder}",
"environment": [],
"externalConsole": false,
"MIMode": "gdb",
"preLaunchTask": "C/C++: g++ build active file"
}
]
}


CPE21S4 - SILVOZA
EXPLORER
CPE21...
.vscode
HOA5
vscode
launch.json
tasks.json
Job.h
main.cpp
Node.h
Printer.cpp
Printer.h
HOA5_1
HOA5P1
QueueActivity
Printer.h
Printer.cpp
main.cpp
tasks.json HOA5\...
launch.json
HOA5 > vscode > tasks.json > ...
{
"version": "2.0.0",
"tasks": [
{
"type": "shell",
"label": "C/C++: g++ build active file",
"command": "g++",
"args": [
"-fdiagnostics-color=always",
"-g",
"-std=c++17",
"${workspaceFolder}/*.cpp",
"-o",
"${workspaceFolder}/main"
],
"options": {
"cwd": "${workspaceFolder}"
},
"problemMatcher": [
"$gcc"
],
"group": {
"kind": "build",
"isDefault": true
}
}
]
}

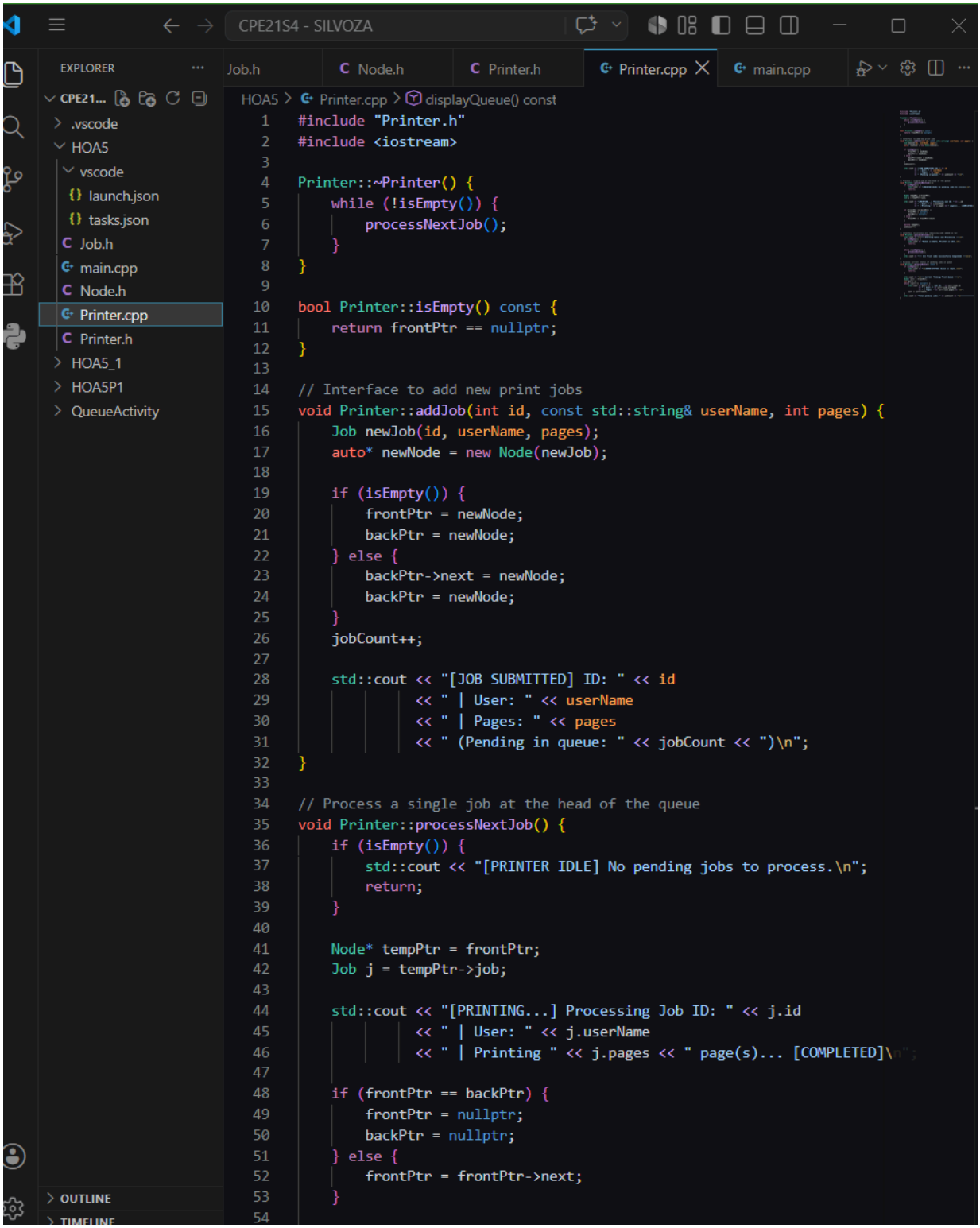

CPE21S4 - SILVOZA
EXPLORER
CPE21S4 - SILVOZA
.vscode
HOA5
vscode
launch.json
tasks.json
Job.h
main.cpp
Node.h
Printer.cpp
Printer.h
HOA5_1
HOA5P1
QueueActivity
Job.h
Node.h
Printer.h
Printer.cpp
main.cpp
HOA5 > Job.h > ...
#ifndef JOB_H
#define JOB_H
#include <string>
#include <utility>
// Create a class called Job
class Job {
public:
int id;
std::string userName;
int pages;
explicit Job(int jobId = 0, std::string name = "", int numPages = 0)
: id(jobId), userName(std::move(name)), pages(numPages) {}
};
#endif

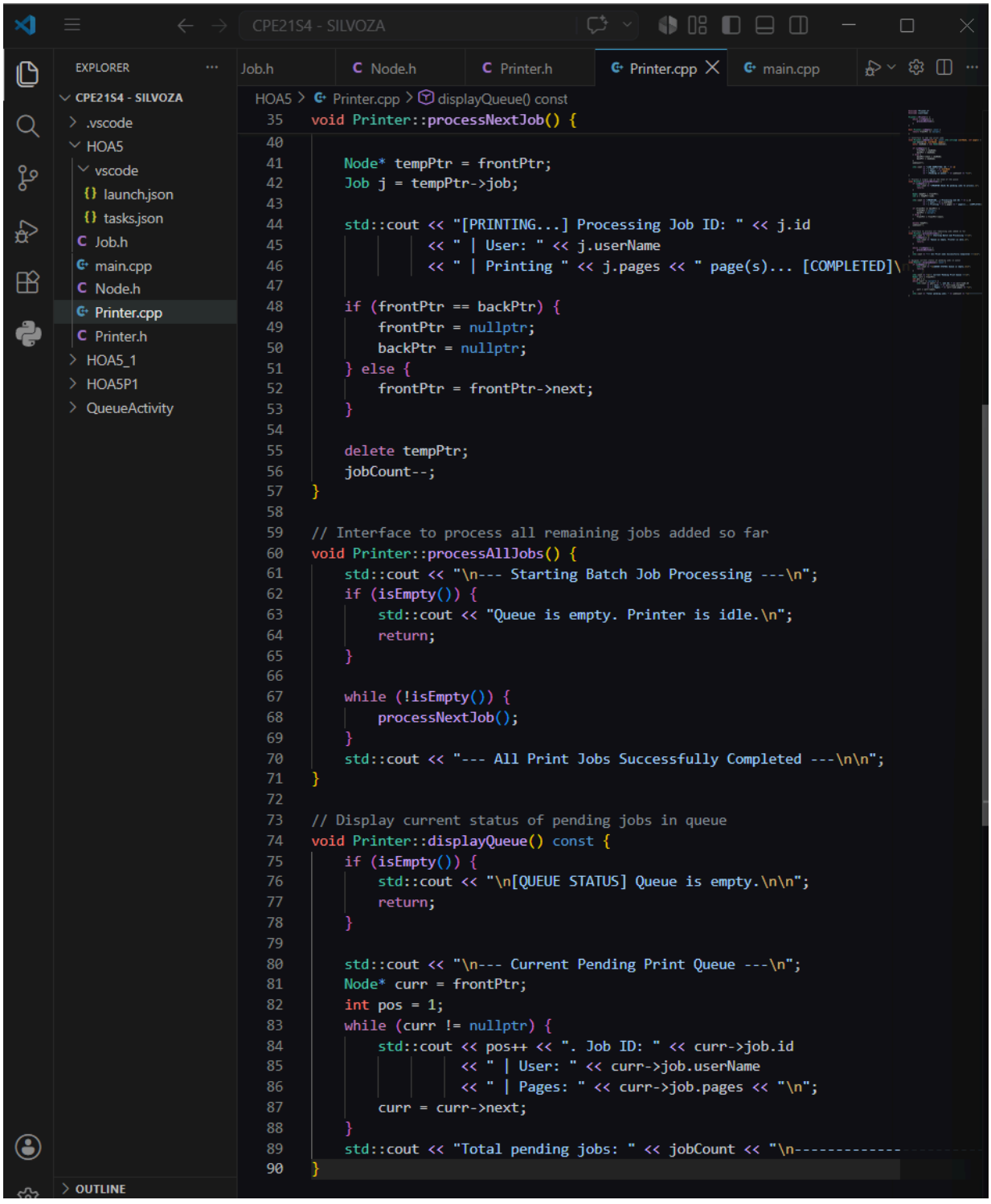

File Edit Selection View Go Run Terminal Help
CPE21S4 - SILVOZA
EXPLORER
CPE21S4 - SILVOZA
.vscode
HOA5
.vscode
launch.json
tasks.json
Job.h
main.cpp
Node.h
Printer.cpp
Printer.h
HOA5_1
HOA5P1
QueueActivity
tasks.json .vscode
Job.h
Node.h
Printer.h
Printer.cpp
HOA5 > main.cpp > main()
#include "Printer.h"
#include <iostream>
int main() {
Printer sharedPrinter;
std::cout << "==========\n";
std::cout << " ILO C: SHARED PRINTER SIMULATION USING QUEUE \n";
std::cout << "==========\n\n";
// Multiple users submit jobs
std::cout << "--- Phase 1: Users submitting initial print requests ---\n";
sharedPrinter.addJob(101, "Alice", 12);
sharedPrinter.addJob(102, "Bob", 5);
sharedPrinter.addJob(103, "Charlie", 25);
sharedPrinter.addJob(104, "David", 3);
// View queue status
sharedPrinter.displayQueue();
// Printer processes jobs one by one
std::cout << "--- Phase 2: Printer begins servicing jobs ---\n";
sharedPrinter.processNextJob(); // Serves Alice
sharedPrinter.processNextJob(); // Serves Bob
// View queue status after initial processing
sharedPrinter.displayQueue();
// New user submits job mid-operation
std::cout << "--- Phase 3: New incoming print request ---\n";
sharedPrinter.addJob(105, "Eva", 8);
// Printer processes remaining batch
sharedPrinter.processAllJobs();
return 0;
}

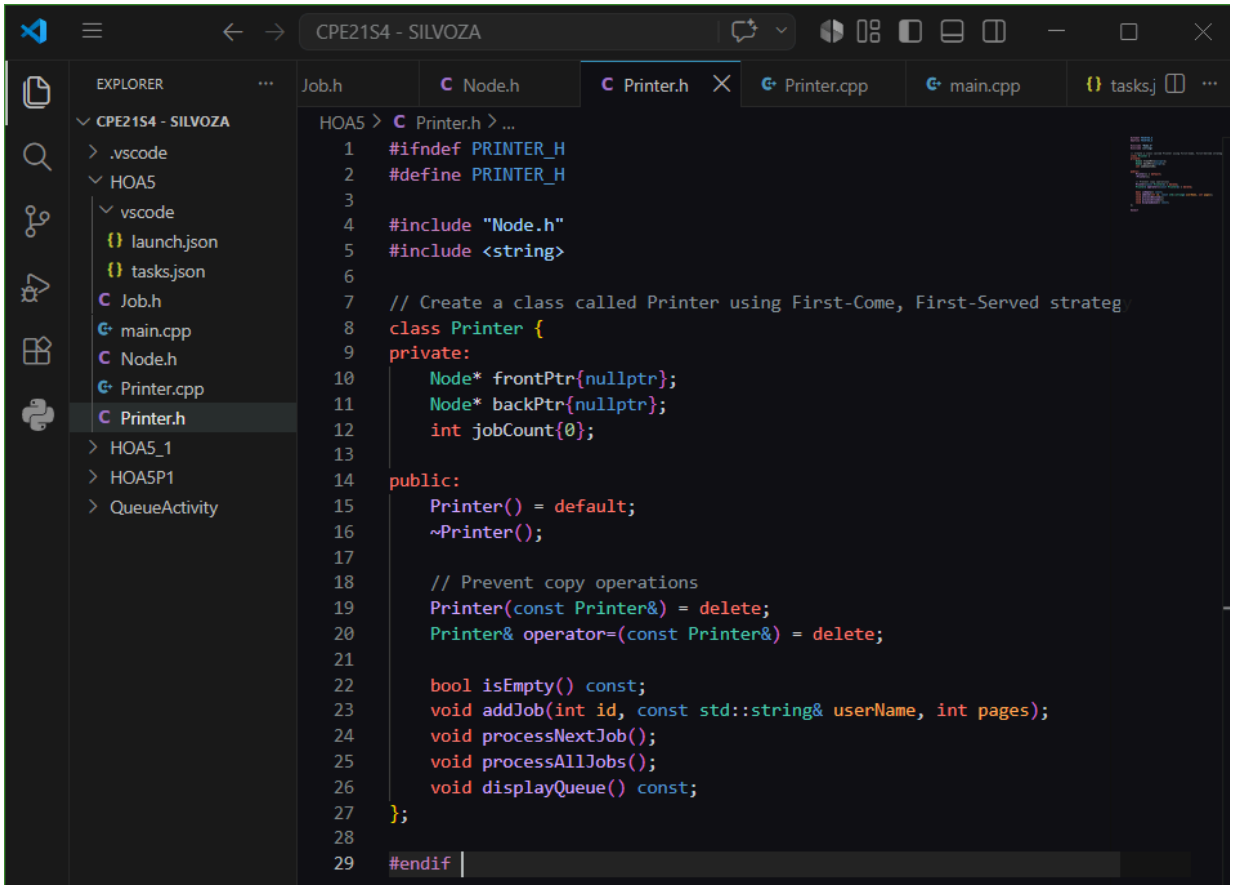

CPE21S4 - SILVOZA
EXPLORER
CPE21S4 - SILVOZA
.vscode
HOA5
vscode
launch.json
tasks.json
Job.h
main.cpp
Node.h
Printer.cpp
Printer.h
HOA5_1
HOA5P1
QueueActivity
Job.h
Node.h
Printer.h
Printer.cpp
main.cpp
tasks.j
HOA5 > Node.h > ...
#ifndef NODE_H
#define NODE_H
#include "Job.h"
// Custom Node for Linked List Queue Implementation (No STL used)
struct Node {
Job job;
Node* next;
explicit Node(Job j) : job(std::move(j)), next(nullptr) {}
};
#endif

COMPUTER ENGINEERING

QUEZON CITY




CPE21S4 - SILVOZA
EXPLORER
CPE21S4 - SILVOZA
.vscode
HOA5
vscode
launch.json
tasks.json
Job.h
main.cpp
Node.h
Printer.cpp
Printer.h
HOA5_1
HOA5P1
QueueActivity
OUTLINE
Job.h
Node.h
Printer.h
Printer.cpp
main.cpp
HOA5 > Printer.cpp > displayQueue() const
35 void Printer::processNextJob() {
40
41 Node* tempPtr = frontPtr;
42 Job j = tempPtr->job;
43
44 std::cout << "[PRINTING...] Processing Job ID: " << j.id
45 << " | User: " << j.userName
46 << " | Printing " << j.pages << " page(s)... [COMPLETED]\
47
48 if (frontPtr == backPtr) {
49 frontPtr = nullptr;
50 backPtr = nullptr;
51 } else {
52 frontPtr = frontPtr->next;
53 }
54
55 delete tempPtr;
56 jobCount--;
57 }
58
59 // Interface to process all remaining jobs added so far
60 void Printer::processAllJobs() {
61 std::cout << "\n--- Starting Batch Job Processing ---\n";
62 if (isEmpty()) {
63 std::cout << "Queue is empty. Printer is idle.\n";
64 return;
65 }
66
67 while (!isEmpty()) {
68 processNextJob();
69 }
70 std::cout << "--- All Print Jobs Successfully Completed ---\n\n";
71 }
72
73 // Display current status of pending jobs in queue
74 void Printer::displayQueue() const {
75 if (isEmpty()) {
76 std::cout << "\n[QUEUE STATUS] Queue is empty.\n\n";
77 return;
78 }
79
80 std::cout << "\n--- Current Pending Print Queue ---\n";
81 Node* curr = frontPtr;
82 int pos = 1;
83 while (curr != nullptr) {
84 std::cout << pos++ << ". Job ID: " << curr->job.id
85 << " | User: " << curr->job.userName
86 << " | Pages: " << curr->job.pages << "\n";
87 curr = curr->next;
88 }
89 std::cout << "Total pending jobs: " << jobCount << "\n----------
90 }


CPE21S4 - SILVOZA
EXPLORER
CPE21S4 - SILVOZA
.vscode
HOA5
vscode
launch.json
tasks.json
Job.h
main.cpp
Node.h
Printer.cpp
Printer.h
HOA5_1
HOA5P1
QueueActivity
Job.h
Node.h
Printer.h
Printer.cpp
main.cpp
tasks.j
HOA5 > Printer.h > ...
#ifndef PRINTER_H
#define PRINTER_H
#include "Node.h"
#include <string>
// Create a class called Printer using First-Come, First-Served strateg
class Printer {
private:
Node* frontPtr{nullptr};
Node* backPtr{nullptr};
int jobCount{0};
public:
Printer() = default;
~Printer();
// Prevent copy operations
Printer(const Printer&) = delete;
Printer& operator=(const Printer&) = delete;
bool isEmpty() const;
void addJob(int id, const std::string& userName, int pages);
void processNextJob();
void processAllJobs();
void displayQueue() const;
};
#endif

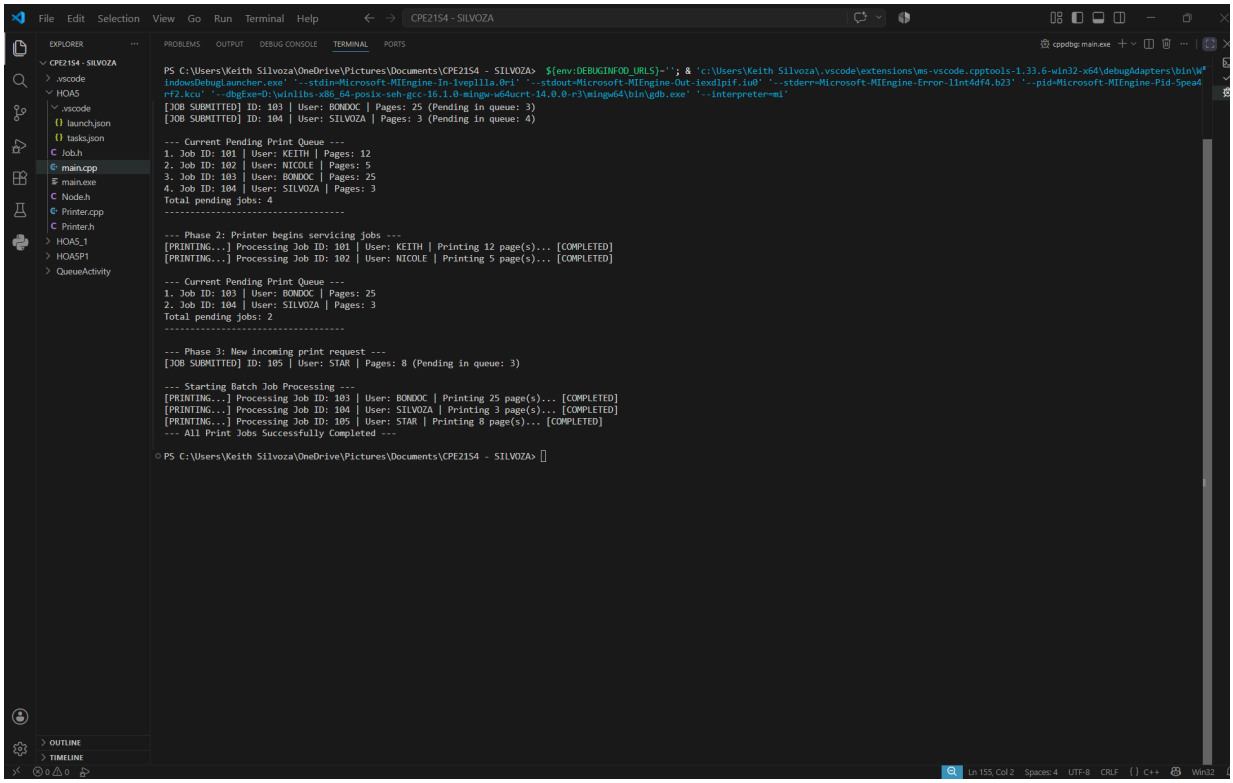
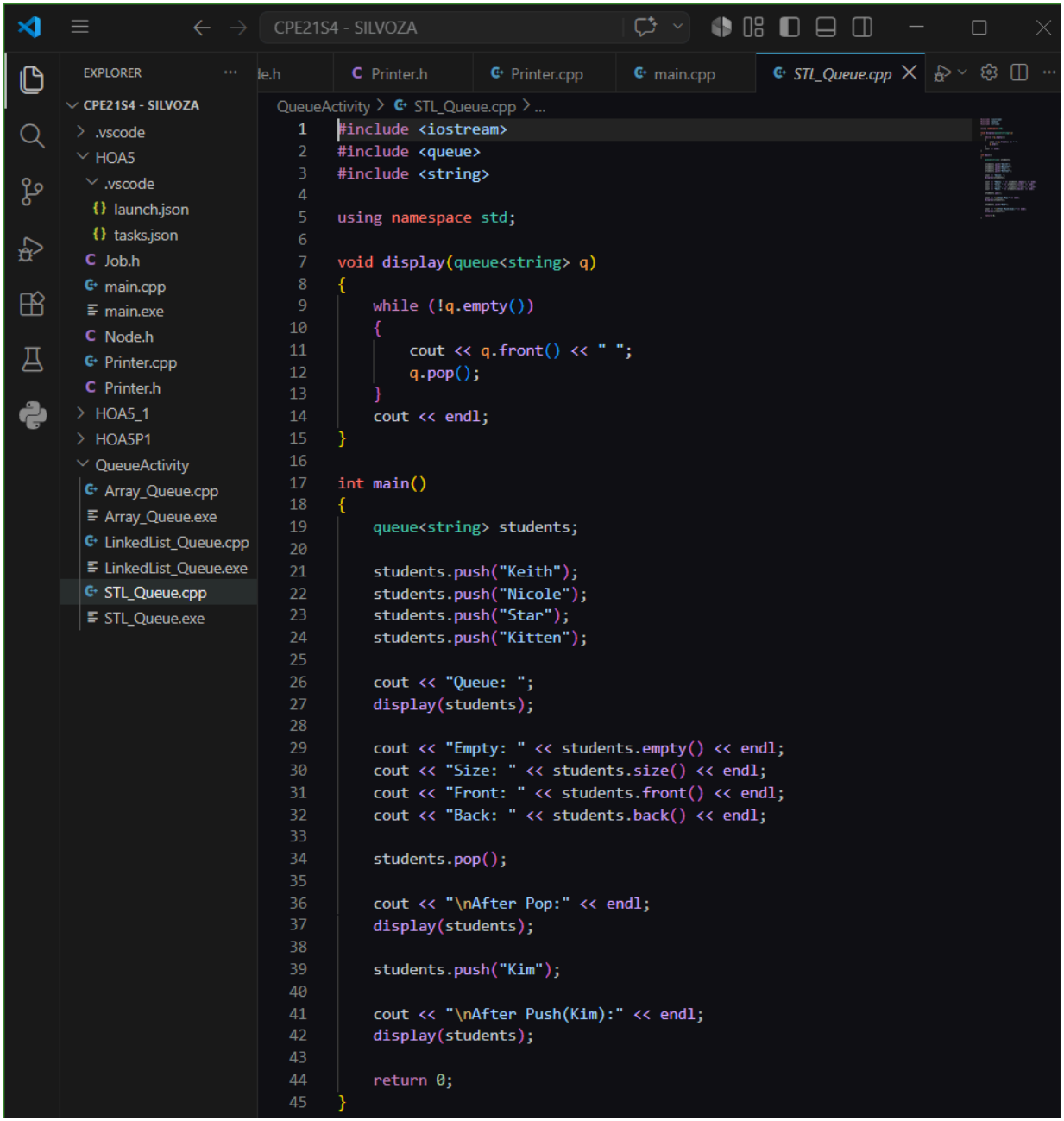


Table 5-1. Queues using C++ STL


CPE21S4 - SILVOZA
EXPLORER
CPE21S4 - SILVOZA
.vscode
HOA5
.vscode
launch.json
tasks.json
Job.h
main.cpp
main.exe
Node.h
Printer.cpp
Printer.h
HOA5_1
HOA5P1
QueueActivity
Array_Queue.cpp
Array_Queue.exe
LinkedList_Queue.cpp
LinkedList_Queue.exe
STL_Queue.cpp
STL_Queue.exe
Printer.h
Printer.cpp
main.cpp
STL_Queue.cpp
QueueActivity > STL_Queue.cpp > ...
#include <iostream>
#include <queue>
#include <string>
using namespace std;
void display(queue<string> q)
{
while (!q.empty())
{
cout << q.front() << " ";
q.pop();
}
cout << endl;
}
int main()
{
queue<string> students;
students.push("Keith");
students.push("Nicole");
students.push("Star");
students.push("Kitten");
cout << "Queue: ";
display(students);
cout << "Empty: " << students.empty() << endl;
cout << "Size: " << students.size() << endl;
cout << "Front: " << students.front() << endl;
cout << "Back: " << students.back() << endl;
students.pop();
cout << "\nAfter Pop:" << endl;
display(students);
students.push("Kim");
cout << "\nAfter Push(Kim):" << endl;
display(students);
return 0;
}

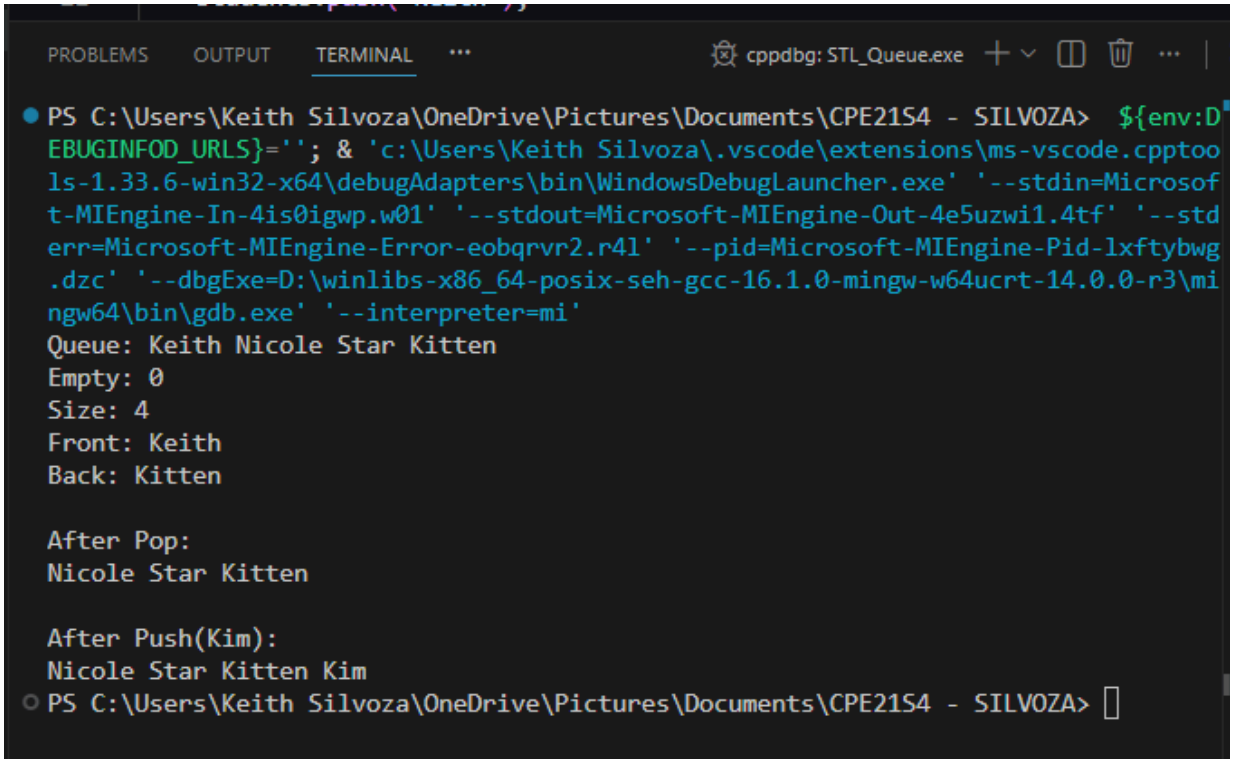
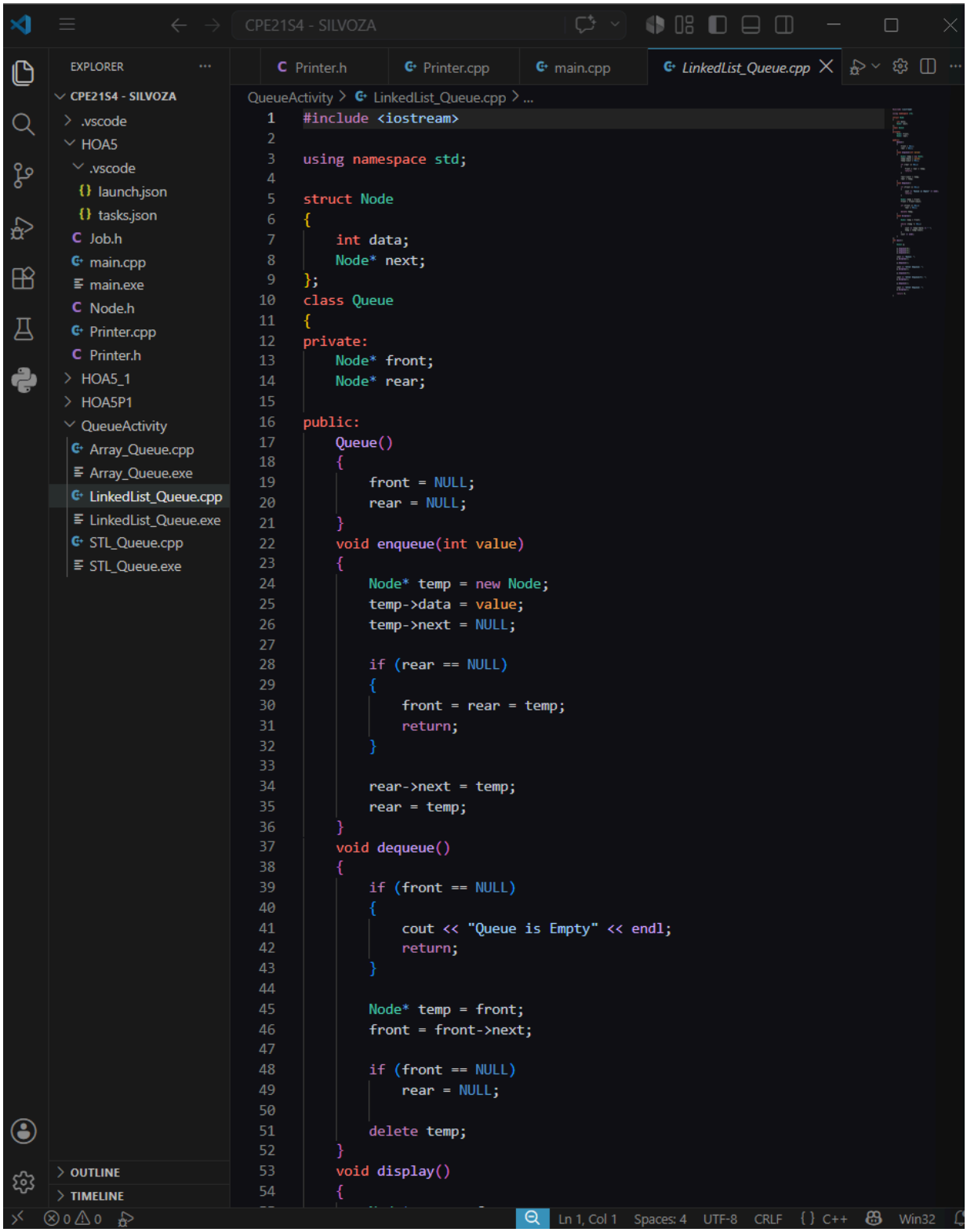
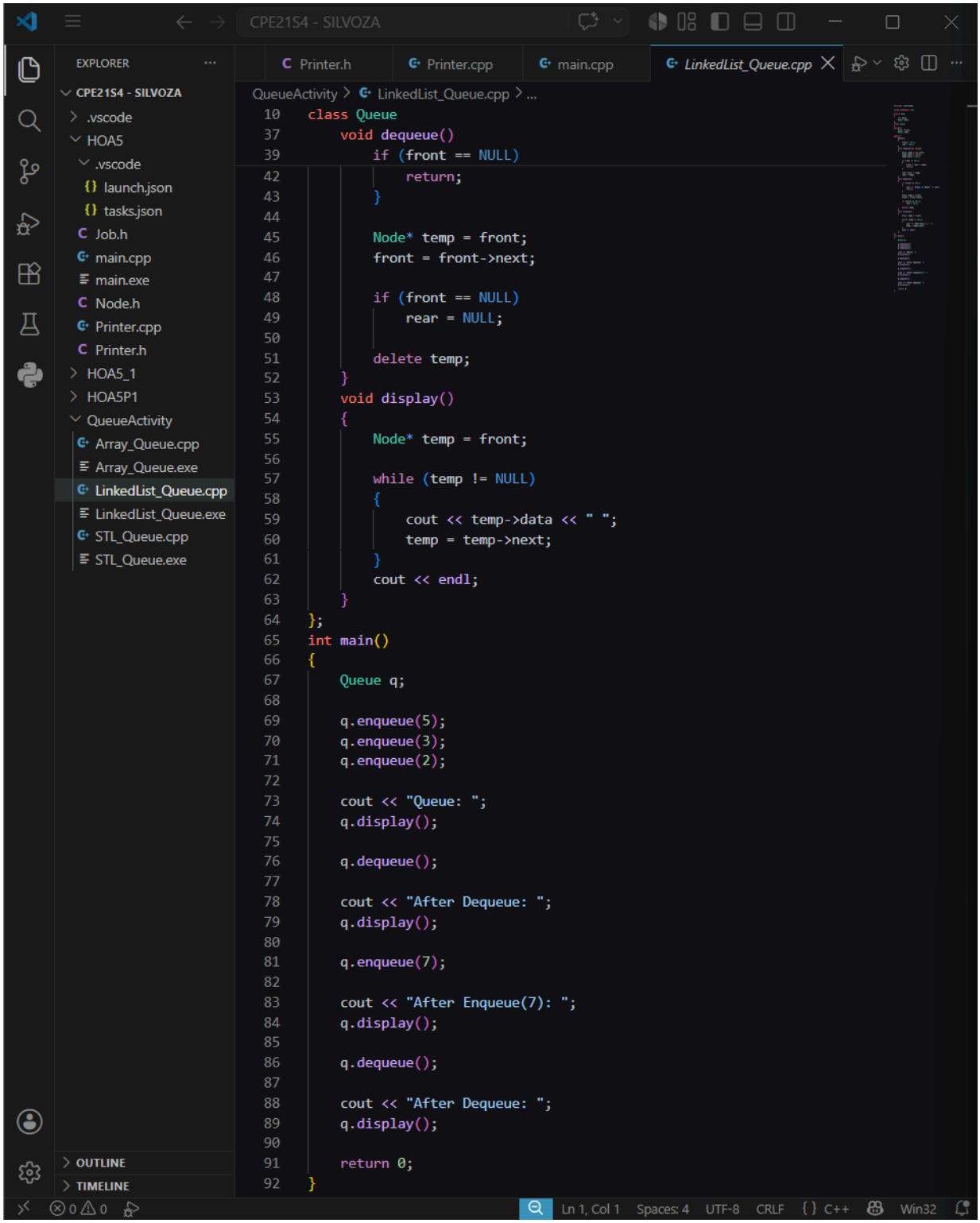


Table 5-2. Queues using Linked List Implementation

COMPUTER ENGINEERING

QUEZON CITY




CPE21S4 - SILVOZA
EXPLORER
CPE21S4 - SILVOZA
.vscode
HOA5
.vscode
launch.json
tasks.json
Job.h
main.cpp
main.exe
Node.h
Printer.cpp
Printer.h
HOA5_1
HOA5P1
QueueActivity
Array_Queue.cpp
Array_Queue.exe
LinkedList_Queue.cpp
LinkedList_Queue.exe
STL_Queue.cpp
STL_Queue.exe
OUTLINE
TIMELINE
Printer.h
Printer.cpp
main.cpp
LinkedList_Queue.cpp
QueueActivity > LinkedList_Queue.cpp > ...
10 class Queue
37 void dequeue()
39 if (front == NULL)
42 return;
43 }
44
45 Node* temp = front;
46 front = front->next;
47
48 if (front == NULL)
49 rear = NULL;
50
51 delete temp;
52 }
53 void display()
54 {
55 Node* temp = front;
56
57 while (temp != NULL)
58 {
59 cout << temp->data << " ";
60 temp = temp->next;
61 }
62 cout << endl;
63 }
64 };
65 int main()
66 {
67 Queue q;
68
69 q.enqueue(5);
70 q.enqueue(3);
71 q.enqueue(2);
72
73 cout << "Queue: ";
74 q.display();
75
76 q.dequeue();
77
78 cout << "After Dequeue: ";
79 q.display();
80
81 q.enqueue(7);
82
83 cout << "After Enqueue(7): ";
84 q.display();
85
86 q.dequeue();
87
88 cout << "After Dequeue: ";
89 q.display();
90
91 return 0;
92 }
Ln 1, Col 1
Spaces: 4
UTF-8
CRLF
C++
Win32

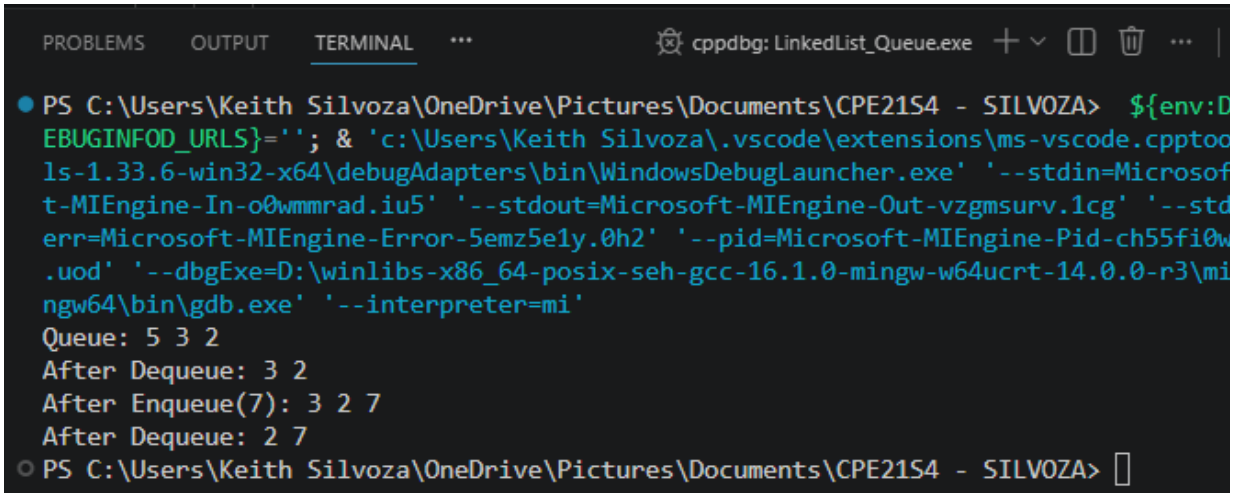
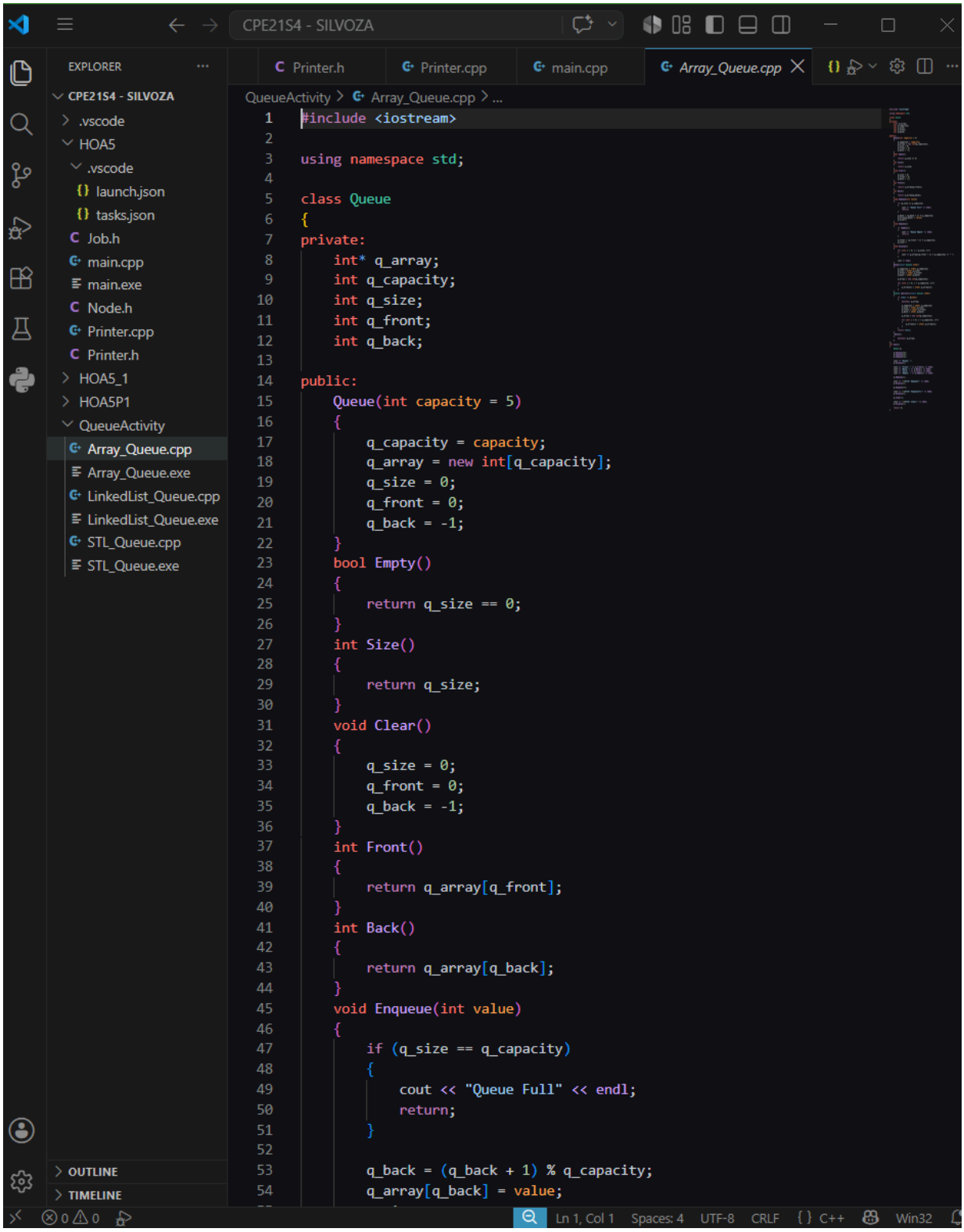
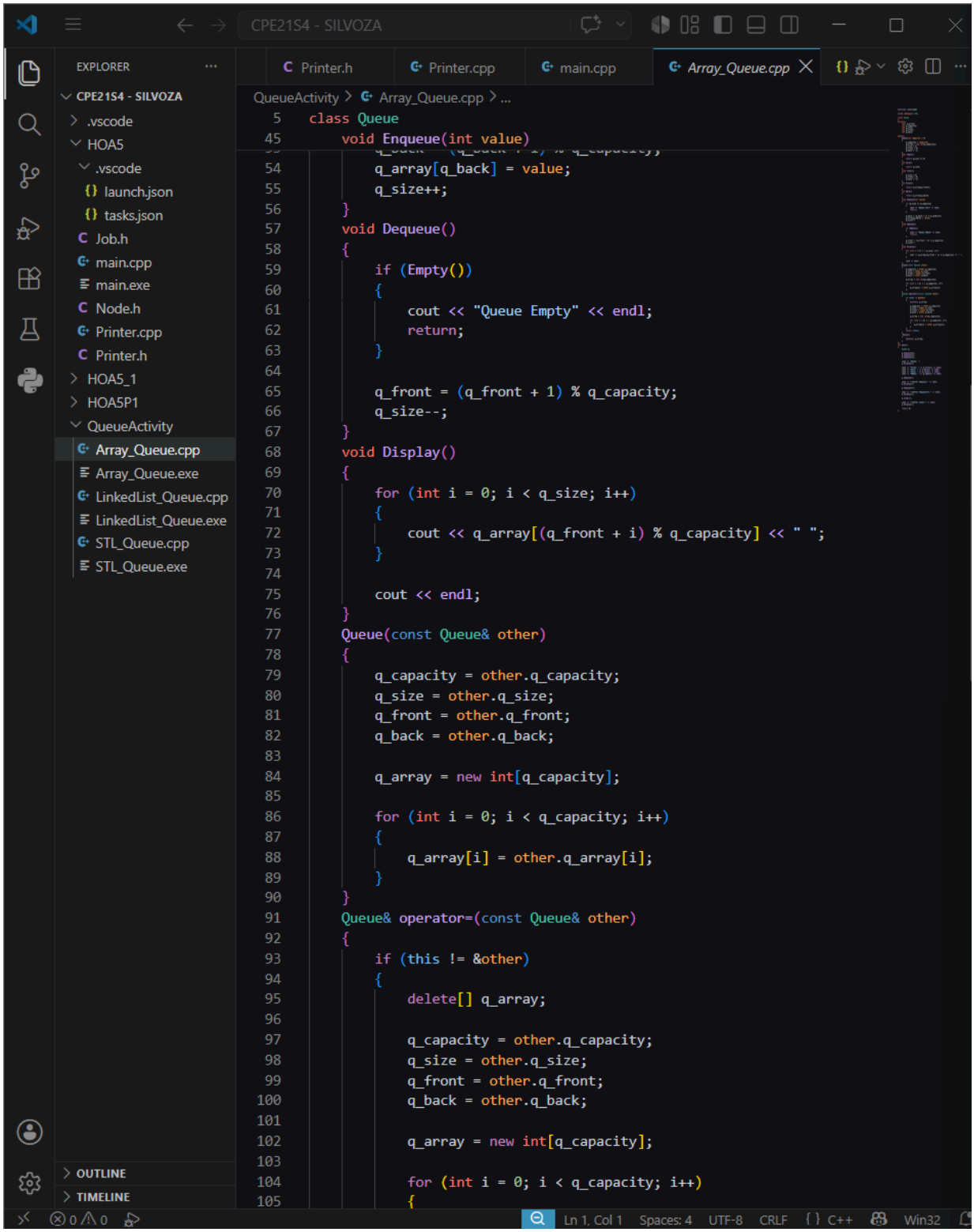
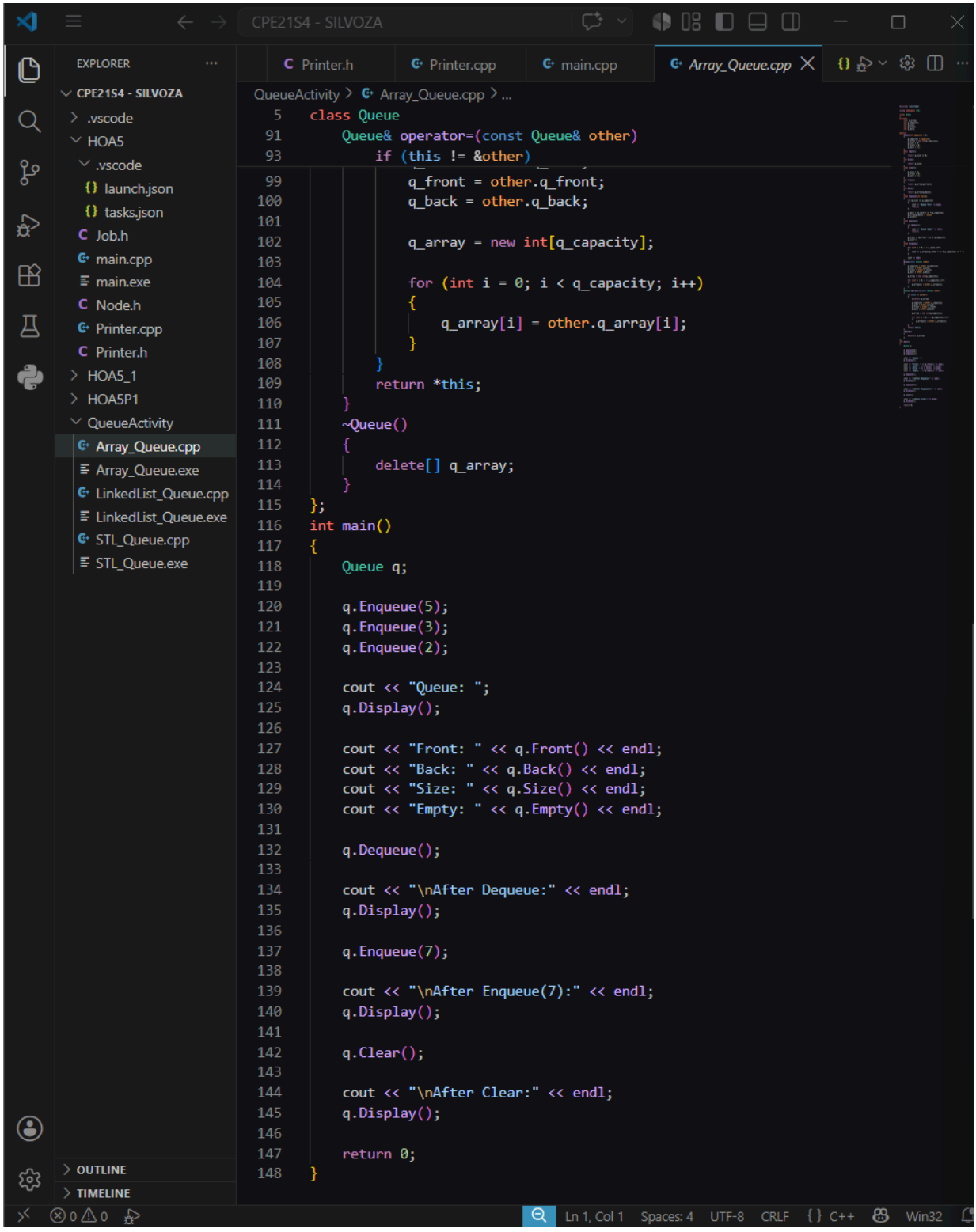


Table 5-3. Queues using Array Implementation


CPE21S4 - SILVOZA
EXPLORER
CPE21S4 - SILVOZA
.vscode
HOA5
.vscode
launch.json
tasks.json
Job.h
main.cpp
main.exe
Node.h
Printer.cpp
Printer.h
HOA5_1
HOA5P1
QueueActivity
Array_Queue.cpp
Array_Queue.exe
LinkedList_Queue.cpp
LinkedList_Queue.exe
STL_Queue.cpp
STL_Queue.exe
OUTLINE
TIMELINE
Printer.h
Printer.cpp
main.cpp
Array_Queue.cpp
QueueActivity > Array_Queue.cpp > ...
#include <iostream>
using namespace std;
class Queue
{
private:
int* q_array;
int q_capacity;
int q_size;
int q_front;
int q_back;
public:
Queue(int capacity = 5)
{
q_capacity = capacity;
q_array = new int[q_capacity];
q_size = 0;
q_front = 0;
q_back = -1;
}
bool Empty()
{
return q_size == 0;
}
int Size()
{
return q_size;
}
void Clear()
{
q_size = 0;
q_front = 0;
q_back = -1;
}
int Front()
{
return q_array[q_front];
}
int Back()
{
return q_array[q_back];
}
void Enqueue(int value)
{
if (q_size == q_capacity)
{
cout << "Queue Full" << endl;
return;
}
q_back = (q_back + 1) % q_capacity;
q_array[q_back] = value;
Ln 1, Col 1
Spaces: 4
UTF-8
CRLF
C++
Win32


CPE21S4 - SILVOZA
EXPLORER
CPE21S4 - SILVOZA
.vscode
HOA5
.vscode
launch.json
tasks.json
Job.h
main.cpp
main.exe
Node.h
Printer.cpp
Printer.h
HOA5_1
HOA5P1
QueueActivity
Array_Queue.cpp
Array_Queue.exe
LinkedList_Queue.cpp
LinkedList_Queue.exe
STL_Queue.cpp
STL_Queue.exe
OUTLINE
TIMELINE
Printer.h
Printer.cpp
main.cpp
Array_Queue.cpp
QueueActivity > Array_Queue.cpp > ...
5 class Queue
45 void Enqueue(int value)
54 q_array[q_back] = value;
55 q_size++;
56 }
57 void Dequeue()
58 {
59 if (Empty())
60 {
61 cout << "Queue Empty" << endl;
62 return;
63 }
64
65 q_front = (q_front + 1) % q_capacity;
66 q_size--;
67 }
68 void Display()
69 {
70 for (int i = 0; i < q_size; i++)
71 {
72 cout << q_array[(q_front + i) % q_capacity] << " ";
73 }
74
75 cout << endl;
76 }
77 Queue(const Queue& other)
78 {
79 q_capacity = other.q_capacity;
80 q_size = other.q_size;
81 q_front = other.q_front;
82 q_back = other.q_back;
83
84 q_array = new int[q_capacity];
85
86 for (int i = 0; i < q_capacity; i++)
87 {
88 q_array[i] = other.q_array[i];
89 }
90 }
91 Queue& operator=(const Queue& other)
92 {
93 if (this != &other)
94 {
95 delete[] q_array;
96
97 q_capacity = other.q_capacity;
98 q_size = other.q_size;
99 q_front = other.q_front;
100 q_back = other.q_back;
101
102 q_array = new int[q_capacity];
103
104 for (int i = 0; i < q_capacity; i++)
105 {
Ln 1, Col 1
Spaces: 4
UTF-8
CRLF
C++
Win32


CPE21S4 - SILVOZA
EXPLORER
CPE21S4 - SILVOZA
.vscode
HOA5
.vscode
launch.json
tasks.json
Job.h
main.cpp
main.exe
Node.h
Printer.cpp
Printer.h
HOA5_1
HOA5P1
QueueActivity
Array_Queue.cpp
Array_Queue.exe
LinkedList_Queue.cpp
LinkedList_Queue.exe
STL_Queue.cpp
STL_Queue.exe
OUTLINE
TIMELINE
Printer.h
Printer.cpp
main.cpp
Array_Queue.cpp
QueueActivity > Array_Queue.cpp > ...
5 class Queue
91 Queue& operator=(const Queue& other)
93 if (this != &other)
99 q_front = other.q_front;
100 q_back = other.q_back;
101
102 q_array = new int[q_capacity];
103
104 for (int i = 0; i < q_capacity; i++)
105 {
106 q_array[i] = other.q_array[i];
107 }
108 }
109 return *this;
110 }
111 ~Queue()
112 {
113 delete[] q_array;
114 }
115 };
116 int main()
117 {
118 Queue q;
119
120 q.Enqueue(5);
121 q.Enqueue(3);
122 q.Enqueue(2);
123
124 cout << "Queue: ";
125 q.Display();
126
127 cout << "Front: " << q.Front() << endl;
128 cout << "Back: " << q.Back() << endl;
129 cout << "Size: " << q.Size() << endl;
130 cout << "Empty: " << q.Empty() << endl;
131
132 q.Dequeue();
133
134 cout << "\nAfter Dequeue:" << endl;
135 q.Display();
136
137 q.Enqueue(7);
138
139 cout << "\nAfter Enqueue(7):" << endl;
140 q.Display();
141
142 q.Clear();
143
144 cout << "\nAfter Clear:" << endl;
145 q.Display();
146
147 return 0;
148 }
Ln 1, Col 1
Spaces: 4
UTF-8
CRLF
C++
Win32

CONCLUSION:

● I learned how queues work and how different implementations in C++ affect their performance and usage.
● The activity was straightforward and helped me understand each queue operation by applying it in code. ● It gave me hands-on experience and improved my understanding of queues through actual implementation.
● This activity strengthened both my understanding of queues and my C++ programming skills. I completed the tasks successfully, but I still need more practice to write cleaner and more efficient code.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **CHATGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance, explanation of queue concepts, and grammar checking.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI was used to help explain queue concepts, provide coding guidance, and assist in debugging. I wrote, tested, and verified the final program myself before submission.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.